



UPPSALA
UNIVERSITET

IT mDA 25 020

Degree project 30 credits

June, 2025

Investigation of OCR Model Performance and Post-OCR Correction Strategies

A Comparative Analysis

Elise Hammarström

Master's Programme in Data Science



Abstract

The act of digitizing text from images and documents is Optical Character Recognition (OCR). This thesis investigates the effectiveness of modern OCR models and post-OCR correction strategies when applied to Swedish historical newspapers from 1818-2018. The aim of the thesis is to provide a thorough understanding of current optimal digitization approaches, both in terms of the OCR process and the post-OCR process where erroneous OCR output is corrected.

Two post-OCR approaches are evaluated: model transferability across different OCR engines and models trained on synthetic data.

Using a dataset of 43,823 newspaper paragraphs, four OCR engines were evaluated: Tesseract, Kraken, DocTR and Idefics. These were selected to display a variety of OCR engines, which includes vision language models, models specifically trained on historical material, and bidirectional long short-term memory text recognition models.

The results show that Tesseract version 5.5.0 overall performs best with 6.7% Character Error Rate (CER) and 22.4% Word Error Rate (WER), followed by Kraken German with 7.6% CER and 26.5% WER. Even though Kraken German consistently struggles with the Swedish character "å", it still outperformed Tesseract in one time period of the data. This, along with the 8-fold increase in CER between the modern period (1980-2018) and historical period (1818-1859), underscores the need for specialized and localized OCR models for Swedish historical material.

The post-OCR model transferability assessment reveals that all OCR models achieve improved results when their outputs are corrected using matching post-OCR models. However some OCR models like DocTR are more compatible for cross-model post-OCR correction than others. DocTR achieved the lowest CER after post-OCR correction step, surpassing both Tesseract's OCR output and Tesseract's post-OCR corrected output.

The synthetic data approach found that a combination of synthetic data from within the domain and outside the domain yielded the best post-OCR results, achieving a 15.9% reduction in word-level errors. However, this synthetic approach is still inferior to post-OCR using real training data, which achieves better results.

Faculty of Science and Technology

Uppsala University, Place of publication Uppsala

Supervisor: Robin Kurtz Subject reader: Gunilla Borgefors

Examiner: Filip Malmberg

1 Acknowledgements

I would like to express my heartfelt gratitude to my supervisor Robin Kurtz and my subject reviewer Gunilla Borgefors, for their support throughout my thesis journey. I am particularly thankful to Robin for allowing me to learn Optical Character Recognition with his guidance and deep knowledge within the field. His mentorship has been invaluable for me in performing this thesis work, and I've learned a great deal about machine learning and computer vision with his guidance. Robin's contribution cannot be overstated, his positive attitude and guidance ensured I could perform this thesis in this (rather comprehensive) scope.

For her guidance, deep knowledge and for making sure I produced an excellent thesis, I sincerely want to thank my subject reviewer Gunilla. Her active support and attention to detail have greatly influenced the work and given me a support-system where I never felt unsure about the next steps or how my text could be more polished.

I also want to thank Anders Hast, for his deep knowledge of OCR and for being an active participant in this thesis from the very start - positively influencing which OCR models might be suitable, teaching me about OCR and reading this report despite his busy schedule with other master's students. His positive attitude and deep expertise, combined with his intellectual curiosity, are a true inspiration.

I would also like to extend my gratitude to the team at KBLab for the opportunity to do my thesis with your team. I've cherished every Tuesday and Thursday and really felt like a member of the team. Thank you!

Last but not least, I want to thank my family and friends for their love, encouragement and unwavering support throughout this process.

Thank you all for your encouragement and support, I am forever grateful for the excellent support system you all contributed to, which made this thesis possible.

Contents

1	Acknowledgements	3
2	Introduction	10
2.1	Aim of the thesis	10
2.2	Scope and limitations	12
2.3	Ethical considerations	12
3	Background	12
3.1	The National Library of Sweden (KB)	13
3.2	Historical newspapers	13
3.3	The OCR-process	16
3.4	Evaluation methods	19
3.5	Related work	20
4	Theory	22
4.1	Neural networks	23
4.2	Convolutional neural networks (CNNs)	25
4.3	Recurrent neural networks (RNNs)	28
4.4	Long short-term memory networks (LSTMs)	29
4.5	Encoder-decoder architectures	31
5	OCR Engines	34
5.1	Tesseract	34
5.2	DocTR	36
5.3	Idefics	38
5.4	Kraken	40
6	The dataset	42
6.1	The annotated ground truth	43
6.2	Typefaces and text formatting distributions	44
6.3	Compression rate	48
7	Methods	49
7.1	Pre-processing	49
7.2	Baseline results	50
7.3	Processing the source material with recent models	51
7.4	Synthetic data generation	54

7.5	Post-OCR training	64
7.6	Technical infrastructure	65
7.7	Evaluation methods	66
8	Results	66
8.1	Comparative performance of recent OCR models	66
8.2	Post-OCR correction model transferability	73
8.3	Training with synthetic data	81
9	Discussion	86
9.1	Comparing OCR model performance	86
9.2	Post-OCR model transferability	89
9.3	Post-OCR with synthetic data	91
10	Conclusion	93
11	Future work	95
Appendix		97
	Annotations in the data	97
	A detailed explanation of each OCR model	98
	Previous OCR results - external results with word lists	104
	Confusion matrices to research question one	105
	Usage of generative AI	109

List of Figures

1	Example of the Fraktur typeface, Barometern 1850	15
2	Example of the Fraktur typeface with inkbleed, Aftonbladet 1827	15
3	Example of the Antiqua typeface, Göteborgs Handels- och Sjöfartstidning 1843	16
4	The OCR-process	16
5	Binarized image with Kraken	17
6	Text area recognition with DocTR	18
7	Unprocessed image, paragraph from Aftonbladet 1827-07-02 .	19
8	A neural network with multiple input nodes, hidden layers and output nodes	23
9	Neural network example	24
10	Convolutional neural network	25
11	Filter, bias and feature map	26
12	Transformation of feature map and max pooling	27
13	Processing through the neural network	27
14	Recurrent neural network	28
15	Unrolled recurrent neural network example	29
16	LSTM architecture	30
17	Word embedding example for the word “Let’s”	31
18	Transformer architecture	33
19	Tesseract version 5 architecture	35
20	CRNN recognition network architecture	38
21	Idefics3 architecture	39
22	Vision encoder and language model architecture for Idefics3 .	39
23	Kraken recognition network architecture	41
24	Paragraphs per period	43
25	Annotation distribution in Ground Truth	44
26	Typeface and formatting distribution	45
27	Distribution of characters with annotations, without annota- tions and overlapping annotations	46
28	Percentage of annotations in entire dataset	47
29	Typeface and formatting distribution per period in the dataset, 1818-1906	48
30	Estimated compression rates of images	49
31	Antiqua font sample	56

32	Antiqua font sample, Google Font Source Serif 4	56
33	Fraktur font sample	57
34	Fraktur font sample, Wieynk Fraktur Font Family	57
35	Swabacher font sample	57
36	Swabacher font sample, OffenbacherSchwabacherCAT Font . .	58
37	Regular font sample	58
38	Regular font, PT Serif	58
39	Comparison of different blur settings	59
40	Gaussian noise background	59
41	Comparison of different text distortion settings	60
42	Comparison of different stroke width settings	61
43	Distribution of blur, stroke width and text distortion for news- paper data	62
44	Distribution of blur, stroke width and text distortion for Wikipedia data	63
45	Overall OCR Performance per OCR Engine	67
46	Example of inaccurately transcribed text	72
47	Example of image paragraph with initials	72
48	Kraken recognition network architecture	98
49	Confusion matrix - top 20 errors. Tesseract v5.5.0	105
50	Confusion matrix - top 20 errors. Kraken-German	106
51	Confusion matrix - top 20 errors. Kraken-Austrian	107
52	Confusion matrix - top 20 errors. DocTR	108
53	Confusion matrix - top 20 errors. Idefics	109

List of Tables

1	OCR Engines on CPU and GPU respectively	34
2	OCR Performance Metrics Across Different Time Periods for Tesseract Models	54
3	Overall OCR Performance Metrics	67
4	OCR Performance Metrics for Period 1818-1859	68
5	OCR Performance Metrics for Period 1860-1899	68
6	OCR Performance Metrics for Period 1900-1939	69
7	OCR Performance Metrics for Period 1940-1979	69
8	OCR Performance Metrics for Period 1980-2018	70
9	Overall OCR Performance - CER - Filtered files	73
10	Overall OCR Performance - WER - Filtered files	73
11	Post-OCR correction model performance on Tesseract output with CER of 0.128 and WER of 0.270. Idefics* had CER 0.086 and WER 0.228 before post-OCR.	74
12	Post-OCR correction model performance on DocTR output with CER of 0.154 and WER of 0.509. Idefics* had CER 0.171 and WER 0.554 before post-OCR.	75
13	Post-OCR correction model performance on Idefics output with CER of 1.582 and WER of 1.934. Idefics* had CER 0.130 and WER 0.194 before post-OCR.	76
14	Post-OCR correction model performance on Kraken German output with CER of 0.117 and WER of 0.299. Idefics had CER 0.095 and WER 0.286 before post-OCR.	76
15	Post-OCR Model's correction on Tesseract - Error type summary	77
16	Post-OCR Model's correction on Tesseract - Top character changes by absolute change	78
17	Post-OCR model's correction on DocTR - Error type summary	78
18	Post-OCR model's correction on DocTR - Top character changes by absolute change	79
19	Post-OCR model's correction on Idefics - Error type summary	79
20	Post-OCR model's correction on Idefics - Top character changes by absolute change	80
21	Post-OCR model's correction on Kraken German - Error type summary	80

22	Post-OCR Model's correction on Kraken German - Top character changes by absolute change. (Double oblique hyphen is abbreviated to "d.o. hyphen")	81
23	Post-OCR correction results with synthetic data. Baseline CER of 0.128 and WER of 0.270.	82
24	Newspaper synthetic's correction on Tesseract - Error type summary	82
25	Post-OCR model's correction on newspaper synthetic - Top character changes by absolute change	83
26	Wikipedia synthetic's correction - Error type summary	83
27	Post-OCR model's correction on Wiki Synth - Top character changes by absolute change	84
28	Original + Wikipedia synthetic's correction on Tesseract - Error type summary	84
29	Post-OCR model's correction on original+wiki synth - Top character changes by absolute change	85
30	Annotation Types and HTML Codes	97
31	OCR Performance Metrics Across Time Periods (Baseline Models)	104

2 Introduction

Optical Character Recognition (OCR) is the conversion of images containing printed, typed or handwritten text into digitized text [1, 2]. Images could be photos of documents, scanned documents, images with subtitles on them or a scenic photograph that contains text to digitize.

While OCR on modern print and font typefaces is to a large extent considered a solved problem, performing OCR on historical documents is still far from a solved problem [3]. Historical newspapers present particular difficulties due to degraded print quality, variable typography and historical spelling variations [4].

The challenge of performing OCR on historical material is especially pressing at cultural heritage institutions, which face the task of digitizing vast collections of historical documents. The National Library of Sweden (KB) maintains a collection of 40 million newspaper pages that require high-quality OCR processing for digital accessibility and searchability, demanding both accurate and cost-effective OCR solutions. As new OCR models continue to emerge, it becomes crucial to evaluate how well these recent systems perform on historical Swedish material compared to existing solutions.

Cultural heritage institutions often possess material previously processed by various OCR systems, making it valuable to understand whether post-OCR correction models trained on one engine’s output can effectively improve another engine’s results. Additionally, the feasibility of training post-OCR correction models using synthetic data remains underexplored for Swedish historical material, which could further enhance the post-OCR correction.

This thesis addresses these challenges by evaluating recent OCR engines on Swedish historical newspapers and investigating both cross-model transferability and synthetic data approaches for post-OCR correction.

2.1 Aim of the thesis

The aim of the thesis is to provide a rigorous understanding of current performance, and approaches to performing OCR on Swedish historical material. The aim of the thesis therefore not only provides OCR performance met-

rics for each OCR engine, but also to determine to what extent cross-model transferability and synthetic data can be utilized in a post-OCR processing step.

2.1.1 Research questions

The thesis aims to explore these research questions:

1. What is the comparative performance enhancement offered by recent OCR models?
2. To what extent can erroneous output from OCR *model A* be corrected by a post-OCR correction model designed to rectify output from *OCR model B*?
3. What is the feasibility and effectiveness of training post-OCR correction models using synthetic data?

Research question one serves as a complementary investigation to gain knowledge of the current state of OCR performance on historical documents, and to generate OCR output used in research question two. This comparative analysis identifies the strengths and limitations of different OCR engines when applied to Swedish historical material.

Research question two explores cross-model transferability, essentially determining to what extent different post-OCR correction models can correct erroneous output from another OCR engine. This addresses a practical need for cultural heritage institutions that may have material processed by various OCR systems that could be further enhanced by performing post-OCR correction.

Research question three examines the potential of synthetic data for post-OCR correction by creating synthetic text images with varied fonts, noise patterns, and distortions that simulate historical document characteristics. This explores whether the performance of post-OCR correction could be further enhanced by training on a larger dataset, that mimics the historical document characteristics found in the source material.

2.2 Scope and limitations

The thesis aims to fulfill the research questions by using open-source models. This choice is motivated by supporting open research. As such, commercial models have not been used for any of the research questions. Furthermore, the goal for research question one is to assess OCR model performance when the models are used out-of-the-box. Fine-tuning the models to the specific source material is therefore not within the aim of this thesis.

For the data exploration of the thesis, about $\approx 43\%$ of the source material is annotated. This means that the ground truth not only contains the text within the newspaper paragraph, but also that the font and text formatting has been noted along with it. As such, an assumption is made that the entire dataset follows the same distribution as the subset that is annotated.

2.3 Ethical considerations

As this thesis explores OCR models, the OCR models may have been trained on data that contains gender, cultural, racial, or socio-economic biases. For example, gender bias might manifest in the OCR model’s output as “...*she took care of the kids*” when the ground truth actually reads “...*he took care of the kids*”, reflecting historical gender role assumptions present in the training material. This is especially important for certain OCR systems that also use context to generate the most accurate description of the text in the image.

As the source material consists of Swedish historical newspapers spanning the years 1818-2018, it is important to recognize that the source material itself reflects the social attitudes and language conventions of its time. Even though this thesis is centered around technical OCR performance rather than content analysis, it is important to acknowledge that these attitudes exist and may be amplified through the digitization process, especially where characters are difficult to recognize.

3 Background

This chapter provides the foundational knowledge necessary to understand the challenges and methodologies involved in using OCR to process historical Swedish newspapers. It covers the motivations for using OCR for digitization

purposes and explains the unique challenges posed by historical newspapers, including their variations in typeface and print quality. The technical aspects of the OCR process are then outlined, and finally, evaluation methods to assess OCR performance are presented.

3.1 The National Library of Sweden (KB)

The National Library of Sweden, Kungliga Biblioteket (KB), collects, preserves and provides access to almost everything that has ever been published in Sweden [5]. This includes printed material, video material and sound. As the National Library has a responsibility to collect and preserve historical records, their vast data collections are often of interest to researchers and others to access and use, and preferably in a digitized format. However, creating and maintaining a digital transcript of historical records is no easy undertaking. This is one of the goals of the data science team *KBLab* at the National Library. With 40 million newspaper pages in KB's collection, the need for accurate OCR processing is evident.

3.2 Historical newspapers

It is challenging to perform OCR on historical documents as the quality can vary significantly. A few of the challenges are:

- Faded print
- Ink bleed
- Variable page layouts
- Different typefaces
- Historical spelling variations [6]

Newspapers are particularly challenging due to their mixture of complex layouts, varying typefaces, size, as well as low print and paper quality [4]. Furthermore, historical documents tend to have spelling variations with acronyms, characters and abbreviations that are not used today. As such, possible dictionaries used in OCR systems to identify words and characters might be incomplete to capture the spelling variations present in historical documents [7]. One example of a character which has been used in early book printing

is the character *ſ* which is a “long-s” and used as an “s”. The “long-s” is very hard to distinguish from an “f” [8].

3.2.1 Typefaces - Fraktur, Antiqua and Schwabacher

In the 19th and early 20th century, European newspapers were mostly printed in Gothic typeface, also known as Fraktur or Blackletter. It is well known that Fraktur typeface is hard for OCR software to recognize [3]. Fraktur as a typeface developed as book producers modeled the gothic typeface for print. This is a typeface which is characterized of thin, high and angular, even broken forms, giving text a compressed appearance. See Figure 1 and Figure 2 for examples of Fraktur typeface. This distinctive style, originating from Gothic manuscript traditions, continued in print through the Fraktur style that became dominant in many European publications [8]. One form of the gothic style is *Schwabacher*, a more rounded Fraktur typeface. Generally, Schwabacher was used where italic formatting was used in the Latin style. Both Schwabacher and Fraktur was used in Sweden until mid 19th century [9].

As a reaction to the gothic style, a new form of writing emerged in Italy during the 15th century - the humanist style, characterized by simpler, cleaner and more rounded forms. This style, also called Antiqua, would become the origin of the Antiqua typeface used in book printing [8].

Sweden started printing books during the 16th century with the help of German book printers, particularly from Lübeck. This resulted in Swedish book printing using Fraktur typeface, which was the dominant typeface even into the 19th century. Antiqua gained popularity during the 19th century and the printing style forms the basis for our modern typefaces [8]. See Figure 3 for an example of Antiqua.

Vi wilja nu nämna några ord äfwen om dessa me-
del. Vanligtvis uppgiwas flera ganska goda. Men de
inskränka sig oftast till dietetiska och sanitära föreskrifter,
såsom till renlighet, måttlighet, sinnesslugg, wälde öfwer
passionerna, sund föda och några rent medicinska preserva-
tiver. Alla dessa medels värde erkänna wi. Men wi tro
derjemte, att de icke äro tillräckliga, synnerligast derföre, att
wißt folk eller snarare wiða kläser af menniskor hwarken
kunna eller wilja iakttaga dem. De förmå icke beherska
sina passioner. De hafwa icke tillfälle att förskaffa sig
sund föda. De kunna icke iakttaga nödig renlighet och
försigtighet. Och dessa menniskor äro jußt de, som genom
sina yttre förhållanden, sitt armöd, sitt oordentliga lefnads-
sätt m. m. i främsta rummet äro blottställda för en dis-
position, tjenlig att genom porer och andedrägt emottaga
kolera giftet wid dess första antändning och utdunstning på
något smittadt ställe.

Figure 1: Example of the Fraktur typeface, Barometern 1850

Bollstedt'st säljes ett wärdert wälkon-
ditioneradt exemplar af Abrégé de l'Histo-
ire de la Bible, par L'Ecuy, med 96 wärdra
kopparst., och 1 serstilt Atlas i fel. af 6
förstärkta korter, 2 delar, stor 8:o, w. 5.
N:o 16: 32 f.

Figure 2: Example of the Fraktur typeface with inkbleed, Aftonbladet 1827

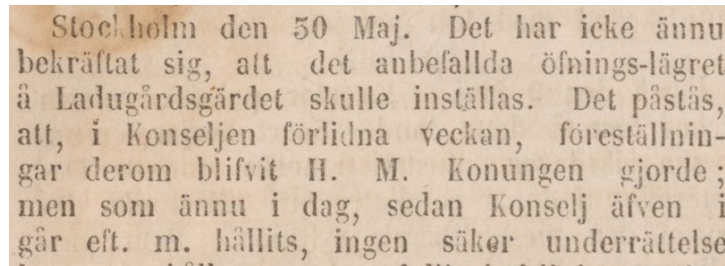


Figure 3: Example of the Antiqua typeface, Göteborgs Handels- och Sjöfartstidning 1843

3.3 The OCR-process

The OCR process generally consists of three steps, pre-processing the image, recognizing characters in the image and lastly post-processing the output to minimize OCR errors. This is seen in Figure 4.

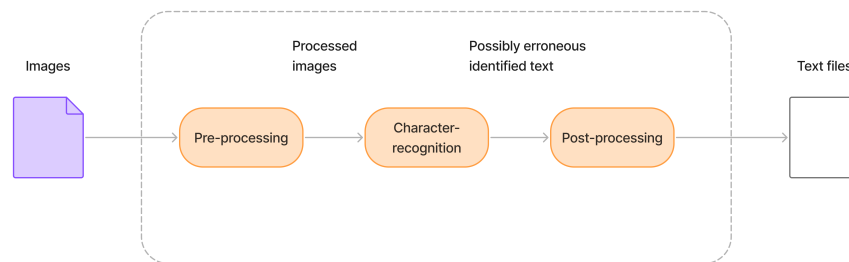


Figure 4: The OCR-process

3.3.1 Pre-processing

The quality of the input images highly affect the output from the OCR systems. One step to mitigate poor image quality is to preprocess the image. The preprocessing steps depends on the OCR engine but common pre-processing steps are *binarization* and *layout analysis*.

Binarization is the process where images are converted into black and white, as seen in Figure 5.

"fartyget till klipporna, på det han må åtkomma några skepps-
"plankor, att dermed reparera sin koja: — den människans.
"som sätter sin grannes hus i brand, för att öfver glöden koka
"åt sig ett ägg." Men lika orsaker frambringa vanligen lika verk-
ningar. Huru mycket skäl därför äfven för oss, att misstänka
den lycka, hvilken man föregifver skola⁷ blifva frugten af den
parlamentariska majoritetens och tryckpressens *allenastyrande*
äfven i det gamla Sverige! Skådom hvad frugter trädet burit i
Frankrike. *Af deras frugt skolen J känna dem.* Aftonbladet
har anat, att något hvar skulle göra dylika reflexioner, och
därför genom några anmärkningar sökt förminska intrycket
och vigten af Karrs artikel. Huru svaga emedlertid dessa an-
märkningar äro, och huru litet de bevisa emot Karr, hoppas
Ins. kunna ådagalägga genom följande påminnelser.

Figure 5: Binarized image with Kraken

The goal of binarization is to separate the text in the image from the back-ground, but also to remove features that introduce noise in the digitized image, like stains [10]. The binarized images are then fed into the *layout analysis* step, where areas containing text are detected in the image [11].

The purpose of pre-processing is to enhance text detection in the images, as text is identified and localized [1]. This positional information then serves as input for the following character recognition step. Figure 6 shows the text area recognition output from the OCR model DocTR.

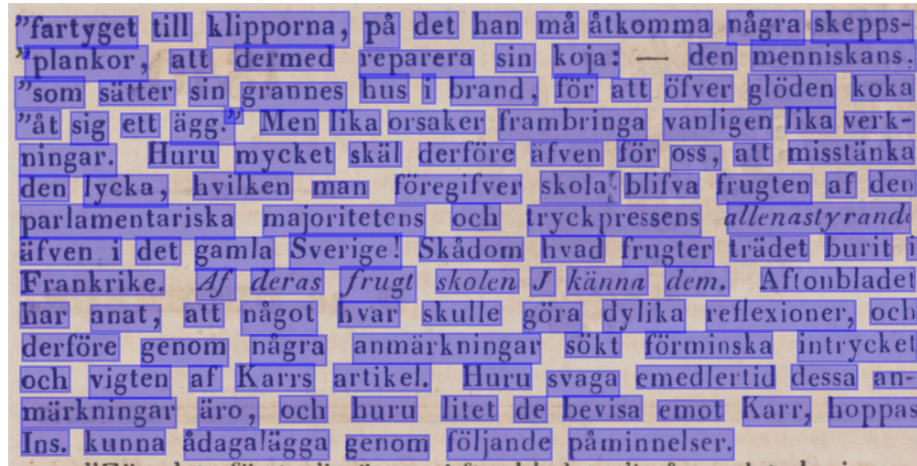


Figure 6: Text area recognition with DocTR

3.3.2 Character recognition

The character recognition step takes the input from the pre-processing, performs character extraction from the image and produces a text file with the identified text [7]. The character recognition step is usually split into two subsequent steps, text detection and text recognition. Text detection aims to localize the text areas in the image which then serve as input for the text recognition step, which aims to recognize the text from identified text regions. It does this by transcribing the visual signals into natural language [1].

3.3.3 Post-processing

However, the text identified by the model during the character recognition step can vary in quality. The material could be difficult for the OCR system to accurately recognize, resulting in poor quality. Poor OCR quality affects the readability and comprehensibility of the material and worsens on-line searchability of the documents [3]. One example of OCR errors is from Aftonbladet 1827-07-02 where the OCR system identified:

“grrac W»tt- och Doddrige hörde tia denna srkr.”

Where the ground truth reads:

“gerne Watts och Doddrige hörde till denna fekt.”

The original paragraph is found in Figure 7.

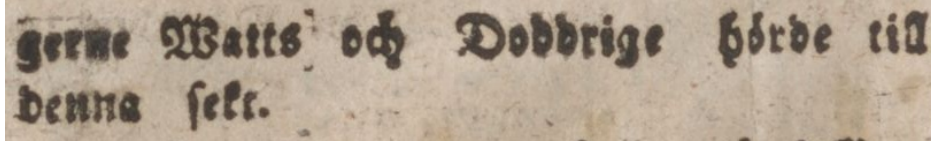


Figure 7: Unprocessed image, paragraph from Aftonbladet 1827-07-02

In order to handle the errors that might occur during OCR, post-OCR correction has emerged as a crucial step in improving the quality of the digitized texts. In post-OCR models, the input is the erroneous digitized text, and the output of the post-OCR model is the corrected text [6].

3.4 Evaluation methods

Character Error Rate and Word Error Rate are used to evaluate model performance.

3.4.1 Character- & Word Error Rate

Character Error Rate (CER) is the minimum number of operations needed to transform the output text into the ground truth. CER is calculated as the division of the sum of all character insertions i , substitutions s and deletions d required to transform the output into the ground truth, which is divided by the n number of characters in the ground truth [12].

$$CER = \frac{i + s + d}{n} \quad (1)$$

Word Error Rate (WER) is calculated in a similar fashion, but with word insertions i_w , word substitutions s_w , word deletions d_i and number of words in ground truth n_w [12].

$$WER = \frac{i_w + s_w + d_w}{n_w} \quad (2)$$

For both CER and WER, a lower percentage is desirable, with 0% representing perfect accuracy.

Another evaluation method that may be used within OCR as a field is Character Accuracy Rate (CAR). CAR is the opposite of CER, where the percentage of correctly identified characters are reported, rather than the erroneous ones. CAR is calculated as $CAR = 100\% - CER$, providing a complementary metric that emphasizes system accuracy rather than error rates. However, CER and WER are the most widely used evaluation methods and will be used within this thesis.

3.5 Related work

OCR technology has evolved significantly with the advances of neural networks and machine learning, but challenges remain in digitizing historical documents specifically. Newspapers pose a particular challenge due to variations in typography, layout and language development [4]. While several studies have explored OCR improvement for historical documents, the combination of using cross-model transferability and utilizing synthetic data for Swedish historical newspapers remains unexplored. Löfgren and Dannélls [6] developed a transformer-based post-correction model, and Dannélls et al. [4] focused on creating a multi-engine approach for post-OCR, but neither explored the potential to use one model’s training data to improve another’s performance.

3.5.1 Previous research on the same source material

Dannélls et al. used a two-engine approach for post-OCR and achieved a 91.64% character accuracy rate (CAR) [4]. The data used in their research is the same data used in this thesis. Their work revealed crucial typography shifts, where the period 1818-1906 showed a 75% Fraktur usage, while 1907-2018 instead contained 75% Antiqua usage, informing our approach to take typography distribution in regard for different time periods. Instead of post-OCR improvement, Brandt Skelbye and Danélls [13] focused on improving the character recognition for the time period 1818–1848 and improved the accuracy by using a CNN-LSTM network, which raised CAR from 82.37% to 97.43%. While these studies focused either on single-period optimization or building a custom post-OCR engine, our work extends their findings by examining how training data from one model can enhance the output for another model.

3.5.2 Cross-model transferability in post-OCR correction

Recent work by Löfgren and Dannélls [6] achieved a 35% reduction in CER using a ByT5 transformer model. A re-implementation of this model is used in our post-OCR correction step. Furthermore, Drobac and Lindén [14] showed similar improvements for Finnish-Swedish documents, reducing CER from 8-13% to 1.7-2.7%. While these studies demonstrated the potential of specialized models in post-OCR, they do not explore cross-model transferability, where one post-OCR model tries to correct the output from another OCR engine. Our approach builds on these findings by investigating how training data from one model can improve another model's OCR output in a post-OCR correction step.

3.5.3 Synthetic data generation

Post-OCR correction has evolved significantly with the integration of synthetic data approaches. Jonathan Bourne explored if training a model purely on synthetic data would improve the post-OCR correction, and especially whether the context of the text to be identified could be included in the post-OCR correction step by providing the context to a language model [15]. This is called Context Leveraging OCR Correction (CLOCR-C). First, Bourne used GPT-4o to generate the synthetic text, centered around 19th-century events as seed prompts and included parameters like year and style. Then given that OCR errors generally have identifiable patterns, the author corrupted the synthetic data by simulating OCR-style errors at specified character error rates. The study used the Llama-3.1 8B instruct model for fine-tuning, which was executed by using a specific prompt to frame the task and socio-cultural context [15]. The research concluded that a fine-tuned language model, trained on synthetic data, can significantly improve post-OCR, with a reduction in CER of 55% and WER of 32%. Another interesting finding is that synthetic data models outperformed models trained on real datasets [15].

While Bourne's approach used large language models to generate synthetic data, other researchers have explored font-based approaches. Gruber et al. proposed a Font-based Synthetic Text Generator (FbSTG) for improving handwritten Cyrillic OCR [16]. One interesting aspect of FbSTG is that it not only can accept fonts, but also has pre-loaded historical backgrounds from

historical papers. Furthermore, the FbSTG simulates handwriting variations while preserving character position data. When testing two OCR systems (Tesseract and TrOCR) on handwritten Kazakh and Russian documents, the authors found that adding synthetic data reduced the CER by approximately 20% and WER by 10%. Their work emphasized the importance of matching the synthetic data distribution (ratios of synthetic data versus dataset size) to the target datasets for optimal OCR performance [16].

This previous research provides insights into 1) the importance of matching the ratio between synthetic data and the target dataset 2) using fine-tuned language models to correct OCR errors based on context. Both of these research papers simulated the OCR errors, which is one approach to enhance post-OCR correction strategies. However, simulating the distortion of the synthetic data material (adding blur, smudging lines etc) has not yet been researched within a Swedish context.

While previous research has demonstrated the effectiveness of both synthetic data generation and cross-model transferability in post-OCR correction separately, there exists a significant research gap in combining these approaches for the application of digitizing historical Swedish newspapers.

4 Theory

This chapter presents the theoretical foundations underlying the neural network architectures and techniques employed in modern OCR systems. It begins with an introduction to neural networks as the fundamental building blocks of deep learning, followed by specialized architectures including convolutional neural networks for image processing and recurrent neural networks for sequential data handling. The challenges of processing long sequences are addressed through Long Short-Term Memory networks, leading to the more recent transformer architectures that form the backbone of current state-of-the-art OCR models. Each section builds upon the previous concepts, providing the theoretical framework necessary to understand the methodologies applied in this research.

4.1 Neural networks

Neural networks are a foundational architecture of deep learning. A neural network is a statistical model generally used to approximate non-linear functions. The goal of the neural network is to approximate some function f^* . One example of a neural network is a classifier, $y = f^*(x)$ which maps the input $\mathbf{x}$ to a category y . To approximate the function f^* , the network learns to map input to outputs through multiple layers of interconnected nodes (the network). The information in the network flows forward, which results in an acyclic graph, with no feedback connections [17]. The information flows from the input node, through the nodes in the hidden layer(s) and finally flows to the output node, as seen in Figure 8. As the information flows forward, neural networks are also called *feedforward neural networks* [17].

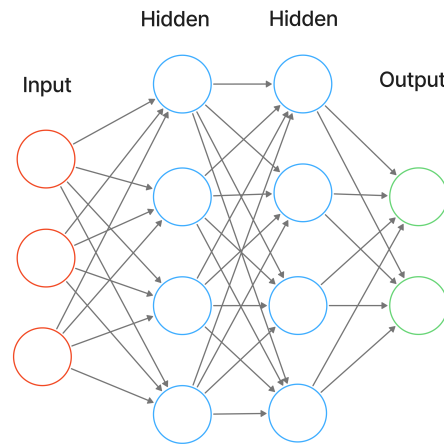


Figure 8: A neural network with multiple input nodes, hidden layers and output nodes

To explain neural networks, a simple example will be made with one input node, two nodes in the hidden layer and one output node. The neural network is visible in Figure 9.

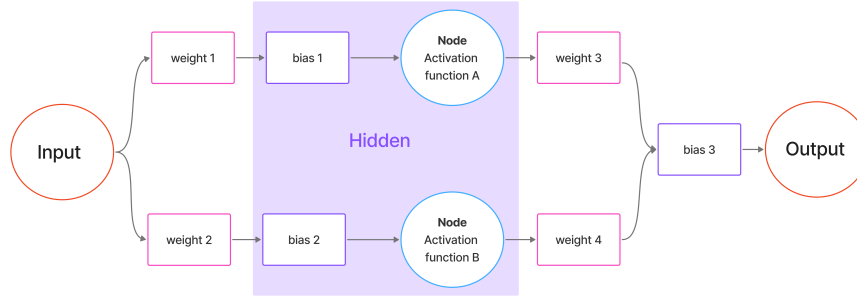


Figure 9: Neural network example

The neural network is first initialized with arbitrary parameter values (values for the weights and biases). The values are then estimated through a method called *backpropagation* when we fit the neural network to the dataset. The neural network defines the mapping $y = f(x; \theta)$ and then learns the values that best approximate the non-linear function, captured by the values in the parameter θ . As such, it learns the values of θ that best capture the function approximation [17].

In our neural network example seen in Figure 9, the input flows through both paths in the network. The input is first multiplied by the weights (weights 1 and 2), and then the biases are added to these weighted inputs. In the nodes in the hidden layer, an activation function is used to approximate non-linear relationships between the input and output, allowing the network to learn complex patterns that linear models cannot capture. Each activation function transforms its input into a new representation. Common activation functions are Softplus, ReLU and Sigmoid. The outputs of these activation functions are then multiplied by their respective weights (weights 3 and 4) and added together along with the final bias term, forming the function approximation that produces the network's output.

Neural networks are usually more complex than this simple example, and usually contain multiple input nodes, output nodes and hidden layers in between, as seen in Figure 8. Furthermore, multiple neural network types exist, where convolutional neural networks and recurrent neural networks are two types of neural networks [17].

4.2 Convolutional neural networks (CNNs)

Convolutional neural networks are specialized neural network that are designed to process input data that has the shape of a grid. One example of input data is time series data, which can be seen as a one dimensional grid with data from different time intervals. Another example of input data is an image, which is a two dimensional grid of pixels.

In traditional neural networks, layers are fully-connected, meaning that each input unit connects to every unit in the next layer. This connectivity pattern continues through all layers in the network. Through this layered structure, every input unit ultimately contributes to every output unit. The weight matrix contains the parameters that encode the strength of connections between units. This results in a network with dense connectivity, where each output is influenced by every element in the input through the learned weight parameters. In contrast, CNNs have sparse connectivity. The sparse connectivity is created when the kernel is smaller than the input size. The kernel (or filter) can be seen as a sliding window, that moves across the input data (the image) processing smaller sections sequentially. For each small section processed, the kernel computes the dot product between its weights and the local region of the input it is currently processing [17]. This can be seen in Figure 10.

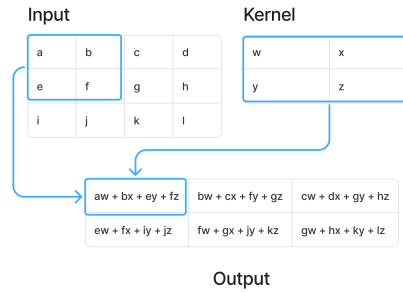


Figure 10: Convolutional neural network

To explain CNNs in detail, a small example is given.

A convolutional neural network example

To explain CNNs, imagine we have an image with 6 x 6 pixels depicting the letter “o”. We want to predict if the letter shown in the image is an “o” or an “x”. To achieve this, the CNN applies a kernel to the input image. Similar to traditional neural networks, before the network is trained, it is initialized with random initial values which are estimated during training using *backpropagation*. This can be seen in Figure 11. After the image has been overlaid with the filter, the dot product between the image and the filter is calculated.

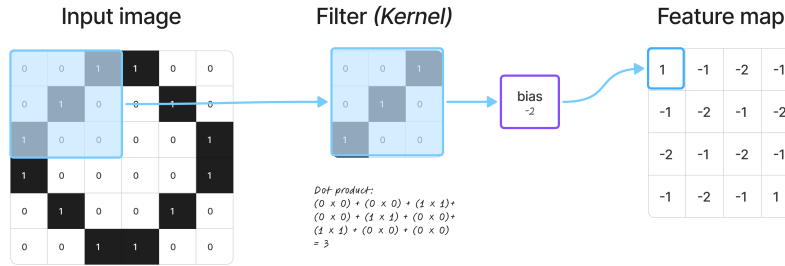


Figure 11: Filter, bias and feature map

The filter then slides over the input image and continues the process until the feature map is filled. The feature map is then used within the activation function ReLU that generates a transformed feature where negative values become zero and positive values remain unchanged. Max pooling is then performed on the transformed feature map, which extracts the maximum number from each section of the feature map, as seen in Figure 12.

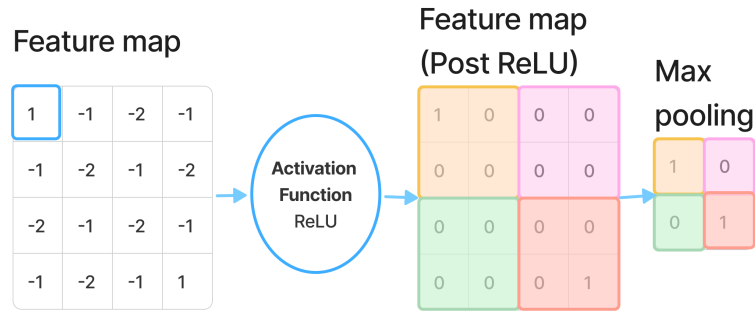


Figure 12: Transformation of feature map and max pooling

The max pooling results are then the input for the neural network, in which weights, biases and activation functions are performed to yield the output, as seen in Figure 13.

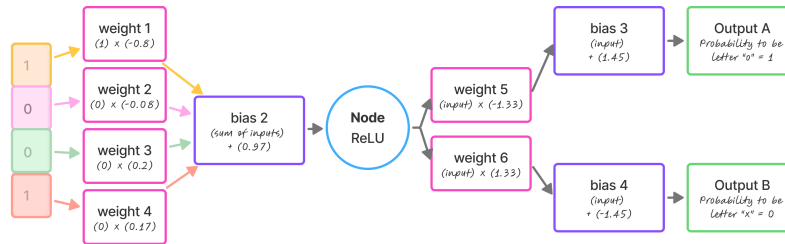


Figure 13: Processing through the neural network

In this example, the input was a 6 x 6 image (36 inputs) that then through the convolution steps got compressed into four inputs to the neural network. The convolutional filter ensures that correlations within the image are maintained, as the filter processes neighboring pixels. Furthermore, through the kernel CNNs can tolerate small changes in the input image.

Beyond this simple example, the kernel processing can be a sliding window of one or more pixels, and the pooling step can also be with average pooling or similar.

4.3 Recurrent neural networks (RNNs)

Recurrent neural networks (RNNs) are neural networks specialized to handle sequential data [17]. The difference between fully connected neural networks and RNNs is that RNNs have feedback loops, where parameter values are shared within the layers of the neural network, as seen in Figure 14.

A recurrent neural network example

Assume we have sequential data on stock prices and wish to predict tomorrow's stock price. Given the scenario where the stock price can either be high or low on a given day, the goal is to use yesterday's and today's stock market value to predict tomorrow's value. An RNN for the simple example is seen in Figure 14, where the input is the sequential data where each data point not only is modified through steps such as weights, biases and activation functions, but also has a feedback loop where previously processed data is taken in as input.

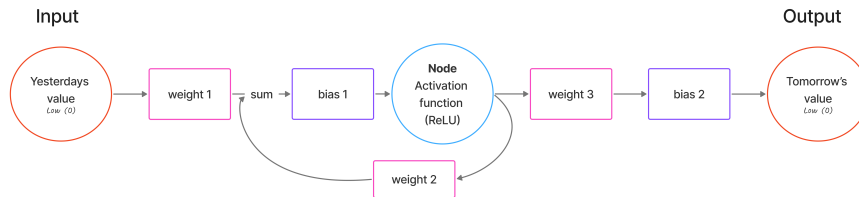


Figure 14: Recurrent neural network

For our simple example, the feedback loop ensures that both yesterday's and today's value is influencing the prediction of tomorrow's value.

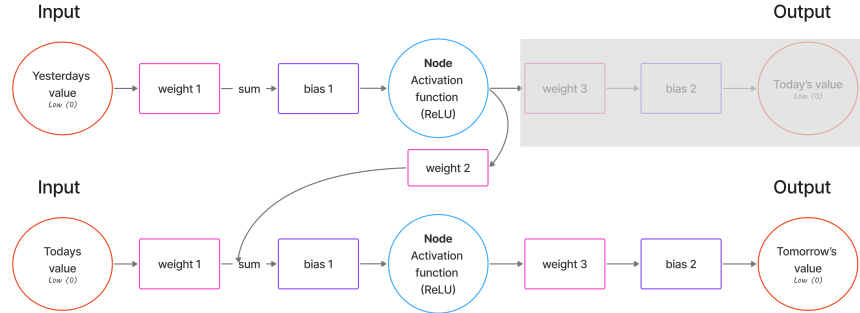


Figure 15: Unrolled recurrent neural network example

The weights and biases are shared across every input in the sequential data. In the unrolled recurrent neural network, the desired prediction for tomorrow's stock market value is seen in the bottom neural network in Figure 15.

However, with long data sequences as input, the *vanishing/exploding gradient problem* may occur. During the *backpropagation* step, gradients for each parameter are calculated using the gradient descent algorithm to find parameter values that minimize the loss function [18]. As the parameters are set through *backpropagation* (flowing backwards), if the $|weight_2| < 1$ the gradient will eventually decay as the gradient becomes so small that early inputs have virtually no influence on the final output. Similarly, if the $|weight_2| > 1$ the gradient explodes, and causes unstable training. The gradient grows exponentially as it is multiplied by $weight_2$ at each time step through time, resulting in extremely large parameter updates that prevent the algorithm from converging to an optimal solution. The *vanishing/exploding gradient problem* is one of the arguments to use LSTMs instead of RNNs.

4.4 Long short-term memory networks (LSTMs)

Long short-term memory networks (LSTMs) is a type of recurrent neural network that is designed to avoid the *vanishing/exploding gradient problem*, while still maintaining relationships between the input data. Instead of using the same feedback loop as regular RNNs, use two separate paths to make predictions about tomorrow. The two paths are the long-term memory and the short-term memory. To address this problem, LSTMs use a more complex

unit design compared to regular RNN architecture. use *sigmoid* and *tanh* activation functions. Sigmoid activation function takes any input from the x-axis coordinate and maps it to a value between zero and one on the y-axis; $\text{sigmoid} = \frac{e^x}{e^x + 1}$. Tanh activation function takes any x-axis coordinate and turns it into a y-axis coordinate between negative one and one; $\text{tanh} = \frac{e^x - e^{-x}}{e^x + e^{-x}}$. The LSTM architecture will now be explained with the visual aid of Figure 16. First, the long term memory is initialized. Observe that the long term memory entirely lacks weights and biases, this ensures that the long-term memory can flow through a series of unrolled units without causing the gradient to explode or vanish. The short-term memory is also initialized and the first step of the LSTM calculates what percentage of the long-term memory that should be remembered throughout the process, this is called the *forget gate*, and adds it to the long-term memory. In the second step, the output is a new long-term memory, passed through the *input gate*. In the last step, the new long-term memory serves as input and creates a new short-term memory, which is the output from the LSTM, passed through the *output gate*.

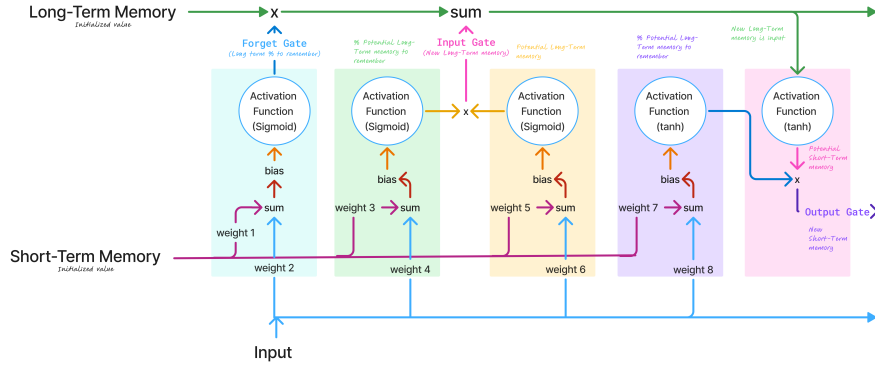


Figure 16: LSTM architecture

These LSTM units are placed in sequence, to process new sequential data while also remembering the previously processed inputs. This gives us a final short-term memory, which is the prediction.

4.5 Encoder-decoder architectures

Transformers are a neural network architecture designed to process sequential data without the recurrence mechanisms of RNNs and LSTMs. Furthermore, the standard transformer does not contain any convolutions. Instead, a transformer consists of an encoder and a decoder [19].

Assume we want to translate the words “Let’s go” from English to Spanish. This is a task that can be done with Transformers. First, the transformer transforms the input words into numbers by using *word embedding*, as seen in Figure 17. Word embeddings use a neural network that has one input for every word and symbol in the vocabulary you want to use. Assume the vocabulary consists of the phrases “Let’s go” and “to go”, as seen in Figure 17.

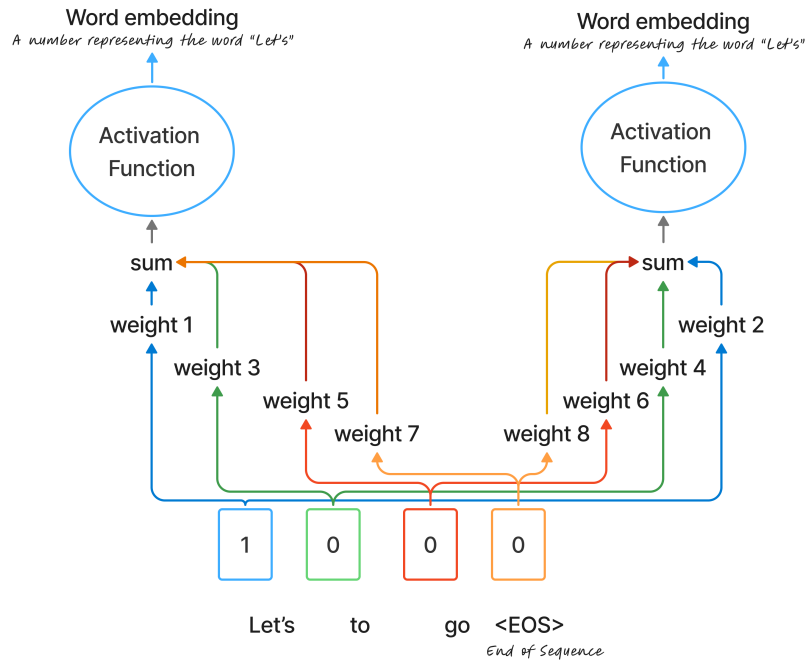


Figure 17: Word embedding example for the word “Let’s”

The word embedding model is a relatively simple neural network, where each input (called a token) is then connected to weights, biases and an activation function. The optimal values for the weights are found through training the network, which uses *backpropagation* to update the parameters. Finally, the output from the word embedding is the numbers corresponding to the word input (called an input embedding). With words, the order must be maintained to understand the meaning of the input. To keep track of word order, transformers utilize *positional encoding*. One way to do positional encoding is to use sine and cosine functions to inject information about the position of tokens in the sequence: $PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{model}})$ and $PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{model}})$, where pos is the position and i is the dimension. These encodings are added to the input embeddings at the bottom of both encoder and decoder stacks [19].

To keep track of the relationship between words, transformers employ a mechanism called *attention*. Attention can be explained by the sentence “It was sunny today, that was nice.” - in processing this sentence the transformer uses attention to relate the word “That” to “sunny”. The transformers does this by using *self-attention*, where (in general terms) the similarity between each word is calculated, including the similarity to the word itself. Once the similarities are calculated, they are used to determine how the transformer encodes each word. Similarly, *multi-head attention* are stacked self-attention units. For our translation example, the model architecture in Figure 18 allows the encoder (to the left) to encode words into numbers, encode the positions of the words, encode the relationships among the words and train in parallel.

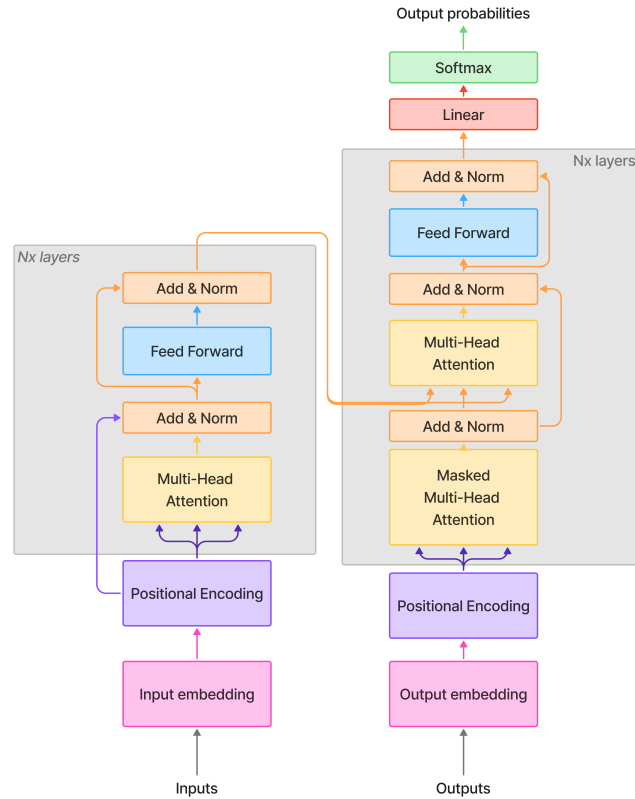


Figure 18: Transformer architecture

The *Add & Norm* step applies residual connections to blend the input of the layer with its output. This is done for continuity, and is followed by a layer of normalization to stabilize and accelerate learning, ensuring consistent scale across the depth of the network. Essentially, the *Add & Norm* step connects the new output with previously processed data. The values of the original data is also retained, which is why the arrows in Figure 18 goes both around and into the next sub-unit [19]. Furthermore, the *Feed Forward* step is a neural network that uses the output from the *Multi-Head Attention* to learn more complex relations between the variables.

With our simple translation example, we have now encoded the phrase “Let’s go” and it is time to decode it into Spanish. Similar to the encoder, now the

output values are embedded, which for our example is a Spanish vocabulary containing “Vamos”. The *positional encoding* is added and again a *Multi-Head Attention* step is used to relate words to each other, but this time, it is a *Masked Multi-Head Attention* step. The “masked” part means it can only look backwards at previous words, not forward at future words. This ensures that when translating “Let’s go” to “Vamos”, the model generates words one at a time from left to right [19]. Furthermore, the *Multi-Head Attention* subunit in the decoder receives input from the encoder, which allows the decoder to look at all parts of the input sentence “Let’s go” when deciding which Spanish word to generate next. Similarly to the encoder, the decoder has a *Feed Forward* network to learn more complex relations between the variables and each unit is wrapped with an *Add & Norm* step. After passing through all six decoder layers, the output goes through a linear transformation and *Softmax* function to produce probabilities for each word in the target vocabulary, predicting “Vamos” to be the Spanish translation of “Let’s go”.

5 OCR Engines

OCR engines used and/or referenced in this thesis are Tesseract, DocTR, Idefics and Kraken.

Some of these models run on either CPU or GPU, these can be seen in Table 1

Table 1: OCR Engines on CPU and GPU respectively

OCR Engine	CPU/GPU
Tesseract	CPU
Kraken	CPU
DocTR	GPU
Idefics	GPU

5.1 Tesseract

Tesseract is an open-source OCR engine that was developed by Hewlett-Packard (HP) between 1984 and 1994, with its open source release in 2005

[20]. From 2006 until November 2018, it was developed by Google [21]. Tesseract is one of the most widely used OCR systems [20]. In this thesis, version 5.5.0 is used (released November 30, 2021). Version 5 maintains the core neural network architecture of a deep learning-based LSTM which was introduced in Tesseract version 4, but with additional improvements and refinements [22]. The architecture consists of multiple sequential components that process text line images into recognized text. The architecture of Tesseract version 5 is shown in Figure 19.

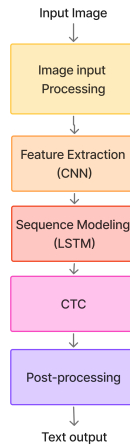


Figure 19: Tesseract version 5 architecture

The input processing component takes the raw image as input and segments it into text lines. Tesseract can work with text lines that are either normalized or unnormalized. In a normalized text line setting, the input undergoes a standardization process that makes the recognition easier and more consistent - baselines of the text are identified (i.e. lines of text), skew is corrected and the height of the letters are scaled to a consistent height. This ensures that regardless of font size or character placement, the baseline is perfectly horizontal and all character components are consistent. In an unnormalized text line setting, it is the raw text line without these geometric standardizations applied. After the input has been processed, the output is sent to the feature extraction component.

The feature extraction component consists of a CNN that processes the image data, by dividing the image up into small patches and analyzing visual information from each patch. Then the sequence modeling component uses bidirectional LSTM networks to process the feature sequence derived from the CNN. The bidirectional approach allows the network to consider both previous context and future context when recognizing each character.

For aligning the network’s output with the actual text, Tesseract uses Connectionist Temporal Classification (CTC). CTC aligns the network’s predictions with the text sequences, ensuring that even though the neural network outputs a matrix every timestep, it is properly aligned with the text from the image. This enables the model to recognize sequences at once instead of requiring individual character segmentation, which improves efficiency and accuracy.

5.2 DocTR

The company Mindee developed and maintains DocTR, a turn-key OCR system powered by Tensorflow 2 and PyTorch. DocTR performs both text detection and text recognition. For each step, DocTR has various model selections. DBNet and CRNN_VGG16 are two such models, used for text detection and text recognition respectively, and will be explained here. The other model selections can be found in Appendix 11.

The text detection component uses DBNet (Differentiable Binarization Network), which addresses the limitations of traditional fixed-threshold binarization. Instead of applying a single threshold across the entire image, DBNet generates an adaptive threshold map where each pixel has its own optimal threshold value. This approach is particularly effective for documents with varying lighting conditions, background complexity, or text contrast levels which might be present in the source material [23].

Text recognition models

CRNN is an end-to-end sequence-to-sequence model, which is a hybrid convolutional recurrent neural network. This is visible in Figure 20. First, the image is fed to the convolutional layer that extracts the key visual information and outputs the feature map. The recurrent layers take the feature map and process it sequentially. The recurrent layers are usually LSTMs and be-

fore the LSTMs there usually is a max pooling step. The transcription layer then takes the sequence of outputs that the RNN generated and maps it onto the actual text sequence, which it does by Connectionist Temporal Classification (CTC) [24]. CRNN is designed as inherently good for sequential data and shows a decent level of robustness towards minor distortions.

The architecture of CRNN consists of both convolutional and recurrent layers. The input image is standardized to a height of 32 pixels. Then the VGG-based convolutional layers (without fully-connected layers) extracts the visual features. The convolutional component consists of multiple convolutional layers with 3x3 kernels and max-pooling layers to reduce the visual features to the most important ones. This outputs a 2D feature map that captures the visual patterns present in the image. Then the feature sequence extraction converts this 2D feature map into a 1D feature sequence by extracting features column by column from left to right, resulting in one vector for each column. Then the deep bidirectional LSTM (typically 2 layers with 256 hidden units each) processes each sequence and captures contextual information from each vector. The Connectionist Temporal Classification (CTC) converts the per-frame predictions into a final character sequence, and as such it handles the alignment between the input sequence and the output labels. Lastly, the CTC decodes the output in a post-processing step.

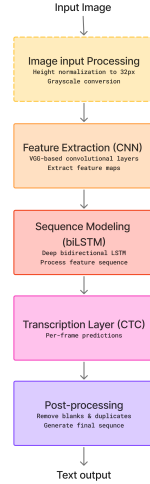


Figure 20: CRNN recognition network architecture

5.3 Idefics

Idefics, a model developed by *HuggingFace*, is an open multi-modal model that accepts both images and text as input and returns text outputs. This thesis employs Idefics3, which improves upon Idefics1 and Idefics2, in which Idefics3 has improved capabilities in OCR, document understanding and visual reasoning. The model can describe visual content in the images, create stories grounded in multiple images as well as answer question about images. It can also act as a pure language model, if no visual inputs are provided to the model [25]. As Idefics can reason about both images and text, it is a Visual Language Model (VLM) [26]. Furthermore, its parent models are Sigmoid Loss for Image Pre-training with 400 Million parameters (SigLIP-SO400M) and `meta-llama/Meta-Llama-3.1-8B-Instruct` [25]. The architecture of Idefics is seen in Figure 21.

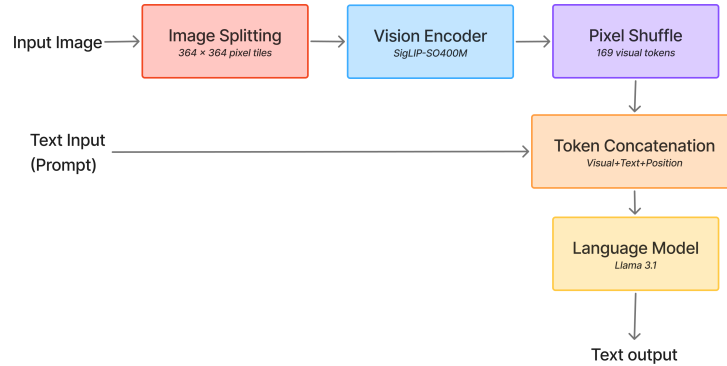


Figure 21: Idefics3 architecture

Idefics splits the input image into 364 x 364 image tiles, upon which each tile is processed by the vision encoder SigLIP-SO400M, as seen in Figure 22.

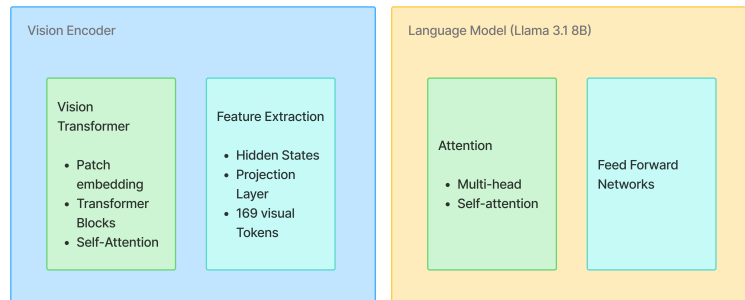


Figure 22: Vision encoder and language model architecture for Idefics3

The Vision Transformer component consists of a patch embedding layer that transforms the raw image into a sequence of embedded patches that the transformer can process. Then the Transformer blocks capture the visual information, where each block (usually 12-24 layers) builds upon the previous blocks, creating increasingly complex feature representations while allowing the model to integrate the information across the entire image. The vision transformer uses self-attention to “attend” to all other image patches, which

enables the model to capture relationships across the entire image, which is crucial to understand complex visual scenes. For the feature extraction component it takes the input from the transformer block which is a sequence of vectors. Each vector represents the image patch and it learns to extract relevant features from these vectors, such as semantic information from the image. Then it has projection layers which transforms the hidden states into a more compact representation, with the purpose of adapting the vision encoder output to be compatible with the language model. The output is 169 visual tokens per image, which stems from that each 364 x 364 pixel tile has been divided into a 13 x 13 grid ($13 \times 13 = 169$). The pixel shuffle component then reduces the dimensionality of the visual representation while preserving spatial information, which is then fed into the language model that is a Llama 3.1 Architecture (8B) which has a transformer stack with multihead- and self-attention components and feed forward networks [25].

Note: Idefics adds “\n” to the OCR:ed output after each processed image patch [25].

5.4 Kraken

Kraken is a turn-key open-source OCR system that is optimized for historical script material. It has a three-stage pipeline consisting of pre-processing (i.e. binarization), segmentation (where layout analysis and line detection is executed) and the text recognition stage. The OCR engine is specifically designed for handling historical documents and various writing systems. Kraken was first published in 2019 and was created by Benjamin Kiessling [11].

The text recognition of Kraken is essentially a rewritten fork of the OCRopus system, where Kraken has been extended with features that enable the processing of most scripts and writing systems, such as right-to-left and vertical writing support, to name a few [11]. What is interesting about Kraken is not only that it is specifically optimized for historical material, but that it also employs a segmentationless sequence-to-sequence approach, which enables Kraken to recognize whole lines of text, without having to split up words into character-level segmentation to then perform recognition on each letter. Instead, the neural network recognizes whole lines of text [27]. This is a clear distinction from other OCR systems, like *Tesseract* [11]. The architecture of Kraken is visible in Figure 23.

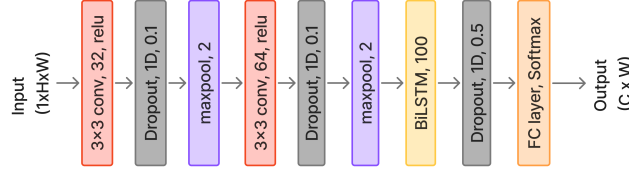


Figure 23: Kraken recognition network architecture

In terms of pre-processing, Kraken performs *binarization* where the images are converted into black and white. For the segmentation stage, Kraken identifies the lines of text by using a “U-Net” to scan the binarized images and find where the lines of texts are located. Then it cleans up this text-highlight-map by smoothing out rough edges (with Gaussian smoothing), ensuring it only keeps the strongest text signals (called hysteresis thresholding), upon which it then creates the lines that follow the text (called skeletonization). This enables Kraken to identify where text is in the image, and also to maintain which order to read the text in.

Once Kraken knows where the text is, Kraken recognizes the text by utilizing a hybrid convolutional and recurrent neural network (CRNN). The convolutional layers spot the visual patterns (like letters) and the recurrent layers handle the sequences, such that words can be properly formed from the visual patterns. This architecture enables Kraken to process an image of a text line and directly output the characters, without having to recognize each letter one by one. Kraken also uses dropout layers, where certain information is dropped out, in order to ensure Kraken is not overly reliant on certain features and patterns. Similarly, it also employs early stopping to ensure Kraken stops the training before it starts memorizing instead of learning [11].

The Kraken OCR engine contains 27 different pre-trained OCR models, where each model is specialized for different languages, scripts, time periods, or document types. These can be found in Appendix 11.

6 The dataset

The reference material is newspapers spanning the years 1818 to 2018. The data from 1818-1906 is publicly available on Språkbanken.¹ The data from 1907-2018 is provided directly from KB, as copyright restricts public access.

The source material consists of 400 pages, selected from 200 different newspapers. From each newspaper, the second and fourth pages have previously been carefully selected. This is because the underlying assumption was that there are generally less advertisements and pictures on these pages, resulting in a dataset with mostly textbased newspaper pages which is more suitable for OCR. From these newspaper pages, each page has been segmented into a paragraph level, where each paragraph has been marked with a border and an ID-number. This data corresponds to 43 826 paragraphs. The dataset with the paragraph identified pages is stored as PDFs, where the paragraph border and the paragraph ID have been created as vector graphics that overlay the original material. The unprocessed material is a JPG-image of the newspaper page.

The dataset consists of 43 826 newspaper paragraphs, from newspapers published during the period 1818-2018. Number of paragraphs processed per period can be seen in Figure 24.

¹Data from 1818-1870: Svenska tidningar 1818-1870, Data from 1871-1906: Svenska tidningar 1871-1906

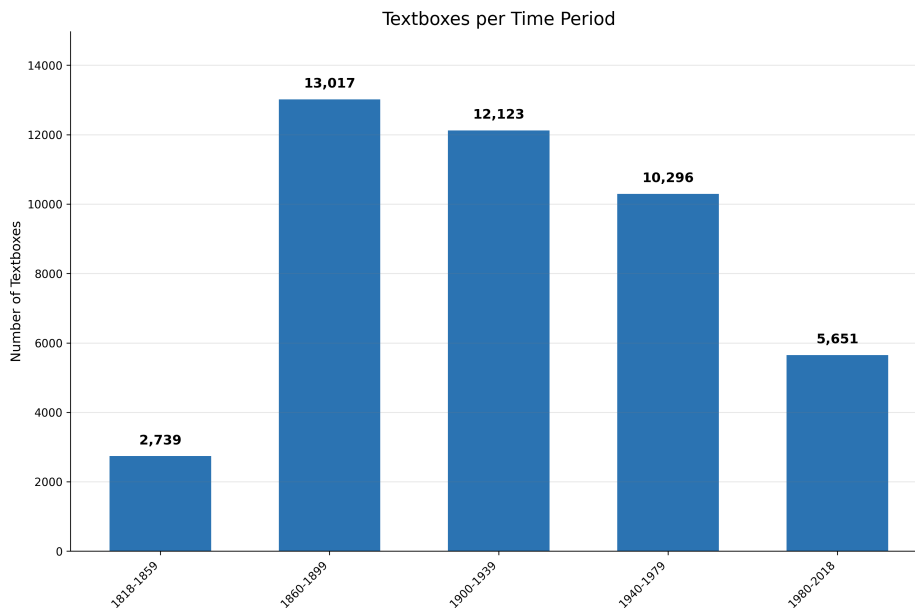


Figure 24: Paragraphs per period

6.1 The annotated ground truth

The following data exploration has been made on the data from 1818-1906, as that data is readily available and annotated. The data from 1907-2018 does not contain any annotations. As such, only the data from 1818-1906 is used in this data exploration. This consists of 17 991 paragraphs ($\approx 41\%$ of the entire dataset).

The annotations used in the ground truth consists of both typeface annotations, text formatting annotations, ligatures and Unicode representations. A complete overview of the annotations used is found in Table 30 in the Appendix. The distribution of annotations (with ligatures excluded) in the dataset is shown in Figure 25.

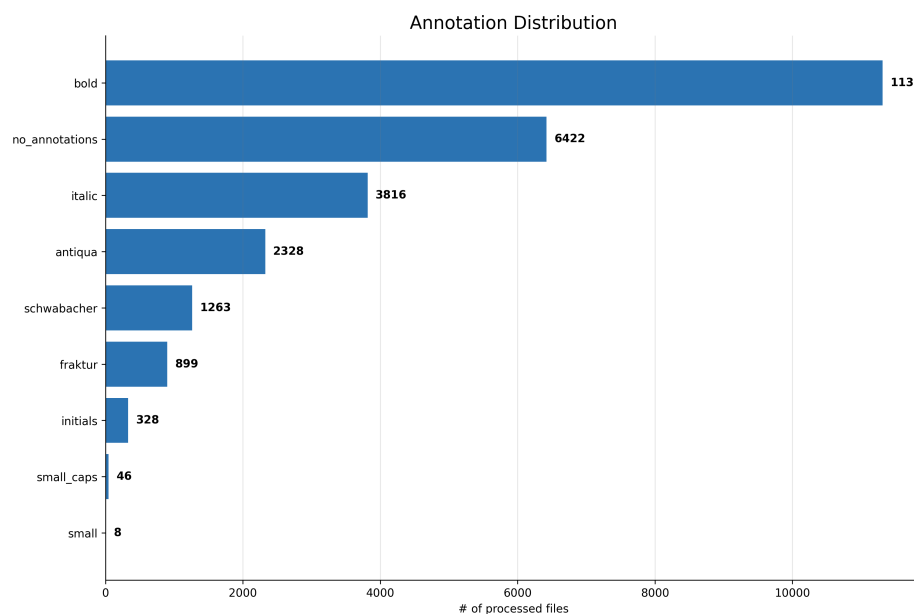


Figure 25: Annotation distribution in Ground Truth

6.2 Typefaces and text formatting distributions

Figure 26 shows the distribution of files containing different typefaces and text formatting.

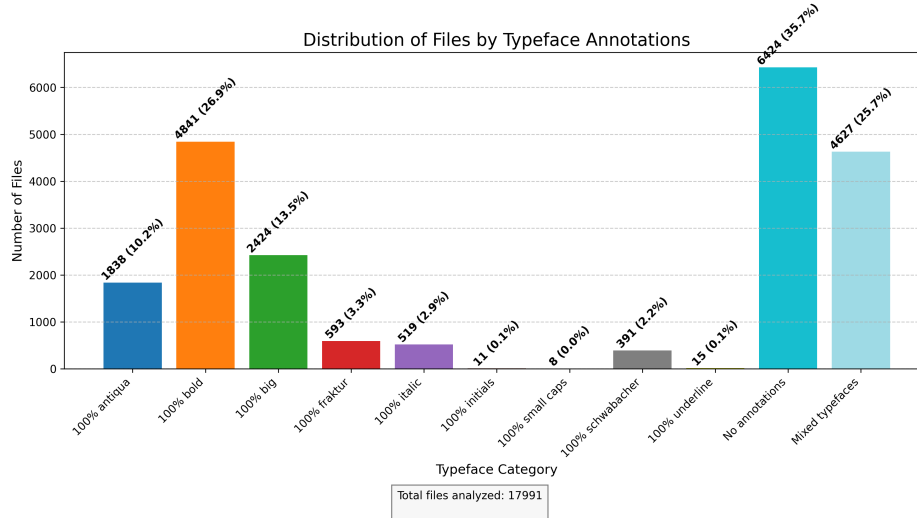


Figure 26: Typeface and formatting distribution

This distribution is important, as it will serve as input for the synthetic data generation along with giving context of the material at hand. The distribution in Figure 26 shows that around 10% of files are entirely written in Antiqua typeface, 3% in Fraktur typeface and 2% in Schwabacher. Furthermore, approximately 37% of the files in the dataset have no annotations at all, and roughly 26% have mixed typefaces, where more than one annotation is present in the ground truth. This gives us insight into what the source material consists of in terms of variations of formatting and typefaces, which might be the reason why certain OCR errors may occur.

Some files contain multiple annotations, such as Fraktur typeface with bold and italic text formatting. These are considered overlapping annotations. The number of files that have overlapping annotations are shown in Figure 27.

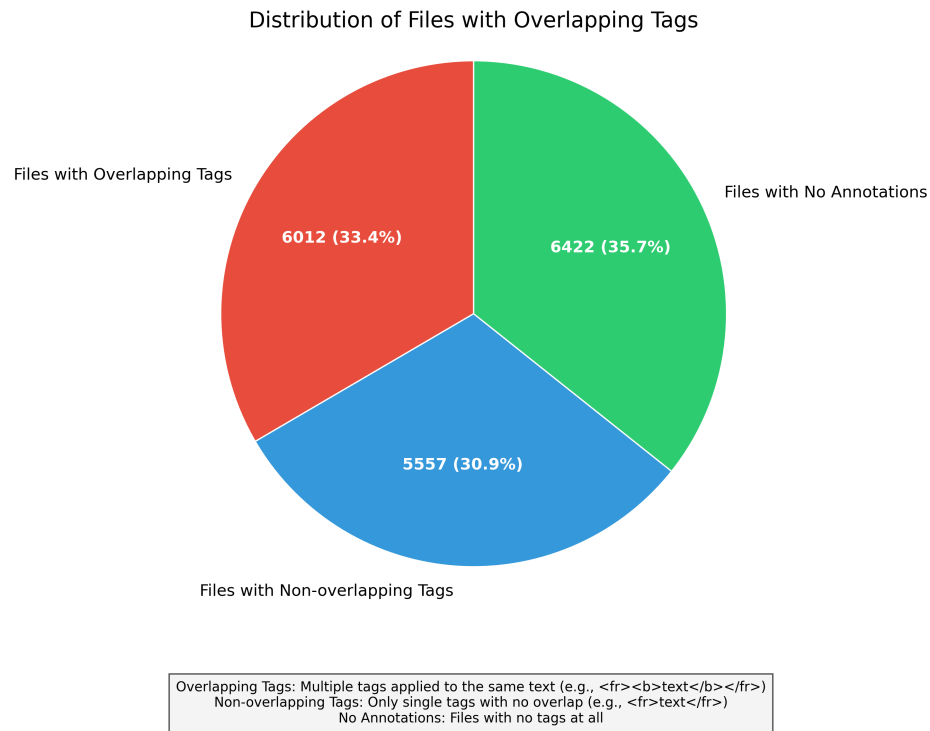
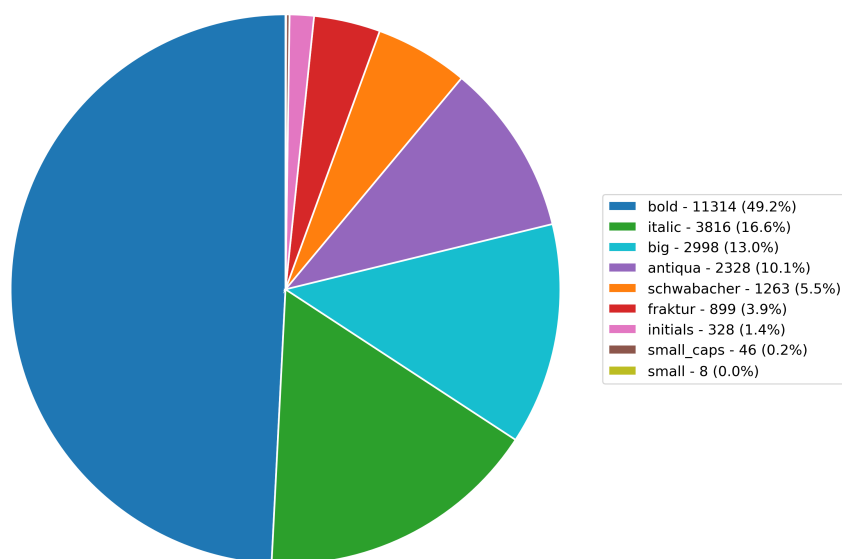


Figure 27: Distribution of characters with annotations, without annotations and overlapping annotations

This serves as an insight into how many characters in the source material that are tagged with multiple annotations. Again, it gives insight into the varying typefaces and formatting present in the dataset.

If one only considers how many characters in the dataset that are annotated, then the distribution of annotations is found in Figure 28. The majority of the characters in the source material are written in bold formatting (49%), followed by italic and big letter formatting at 17% and 13% respectively.

Annotation Type Distribution



Total annotations: 23000

Figure 28: Percentage of annotations in entire dataset

Then the question that arises is which annotations are present during which time period. This can be seen in Figure 29. One insight here is that Antiqua grew in usage in the period 1860-1899 and was barely present in the prior period.

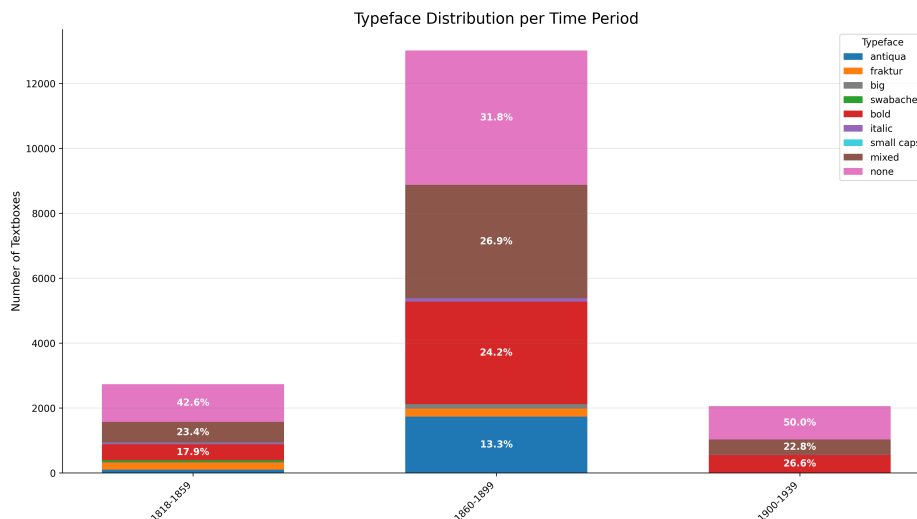


Figure 29: Typeface and formatting distribution per period in the dataset, 1818-1906

6.3 Compression rate

The compression rate analysis was performed by calculating the uncompressed size for each image ($\text{width} \times \text{height} \times \text{color channels} \times \text{bit depth}$), retrieving the actual file size on disk and computing the compression ratio as: uncompressed size divided by the compressed size.

Figure 30 shows that most images are concentrated within compression rates of 7-8 times, with both the average and median at 7.95 times. At this compression level, only 12.6% of the original image information is retained. This relatively narrow distribution suggests consistent digitization practices, though outliers at both extremes warrant further investigation for potential quality issues in the source material.

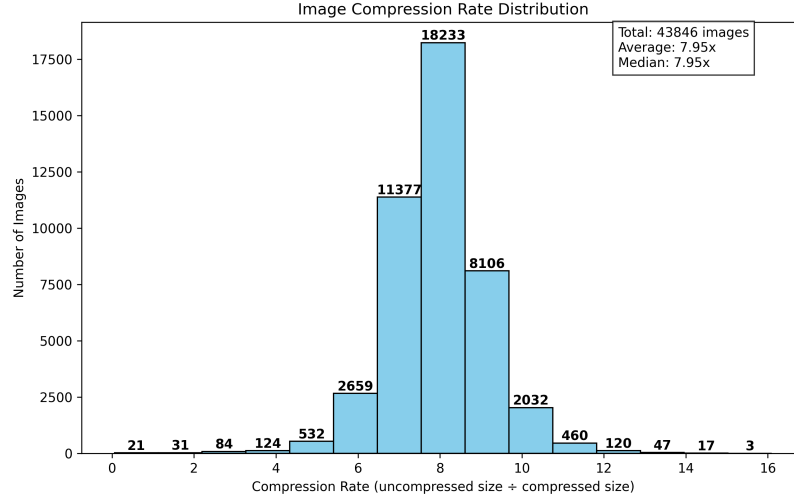


Figure 30: Estimated compression rates of images

7 Methods

This chapter describes the methods used to evaluate OCR performance on historical Swedish newspapers and develop post-OCR correction techniques. It covers the pre-processing steps needed to extract newspaper paragraphs from the dataset and the baseline establishment using Tesseract. The evaluation of recent OCR models is then presented, followed by synthetic data generation techniques to enhance training capabilities. Finally, the development and evaluation of post-OCR correction models is described, including their transferability across different OCR systems.

7.1 Pre-processing

As this master thesis aims to run the reference material through recent OCR models, it is vital to extract each text paragraph from each newspaper page. This to ensure a correct mapping to the transcribed ground truth and to ensure a comparable output to the previous OCR system results. As such, I built a processing pipeline where each paragraph from the original newspaper page is extracted. The data in the dataset consists of one JPG-image of

the newspaper page, and one PDF-image of the newspaper page with vector graphics. The vector graphics is a border around each paragraph, and an ID-number within each border. To extract and correctly map each paragraph to an ID-number in the ground truth, each paragraph and its ID is extracted. The coordinates of the border from the vector graphic is identified and used to extract each newspaper paragraph from the JPG-image of the newspaper page (the page that does not have vector graphics on it). The pipeline generates one JPG-image for each paragraph of the newspaper page, and names the generated JPG-images with the same file naming convention as the ground truth to ensure I easily can match each file with the ground truth.

The dataset also has a ground truth, where the text of the images have previously been manually identified. However, in the instances where the text could not be identified, the transcribers have marked the contents of the file with “@”, signifying illegible text [6]. There also exists instances where the ground truth files are empty. The pipeline removes these instances to prevent incorrect CER and WER scores, such as when OCR accurately recognizes text on images with empty ground truth, which would yield falsely high error rates.

7.1.1 Performing the data exploration

The purpose of the data exploration is to generate an understanding of what the data contains. With this data set, it is interesting to explore the distributions of typefaces, annotations, compression rates and how many newspaper paragraphs each period contains.

7.2 Baseline results

To accurately establish a baseline comparison, Tesseract version 4.1.3 will serve as a primary baseline to compare recent OCR engine results.

Note

The dataset not only has the raw data of each newspaper page, but it also has previous OCR system results, containing CER and WER output from ABBYY FineReader, Tesseract version 4, and a combination of both. The results have previously been generated by Dana Danells and Viktoria Löfgren and are visible in Table 31 in Appendix 11. The results have been achieved by

providing ABBYY and Tesseract with additional word lists. Which have not been done for this study, where the OCR model is assessed as it is, without additional information provided.

7.3 Processing the source material with recent models

This section describes the selection and configuration of OCR models used to process the historical newspaper dataset. The models chosen represent different approaches to OCR and each model requires specific configurations.

7.3.1 Model selection, configuration and evaluation

Model selections include Tesseract, DocTR, Idefics and two Kraken models. These models display a varied range of different OCR systems available. Idefics is a visual language model (VLM), and DocTR, Kraken and Tesseract are all bidirectional LSTM based methods, where Kraken is specifically trained on historical material, Tesseract is (in this thesis) specifically configured to handle Swedish text. The implementation of DocTR uses a flexible binarization threshold to detect the text areas. As the thesis aims to assess out-of-the-box model performance, fine-tuning the models or adding additional language dictionaries for the OCR models that support it, is not within its aim. However, it is important to find the best parameters and/or define a good prompt when applicable.

All models are evaluated on CER and WER metrics, and the metrics have been further analyzed by performing a substitution error analysis, as substitution errors are one component of CER which signifies wrongly recognized characters.

Idefics

As Idefics is a visual language model, the model configuration available is to define the prompt. The following prompt was used:

`If there is text in this image it is in Swedish and then I want you to transcribe it. If no text is visible in the image, leave the output blank. Always have no additional commentary.`

This prompt was chosen as it specifies the language to the model, it gives a clear instruction to transcribe the text in the image, it handles the case where no text is visible and also prevents additional commentary that might

clutter the output, as we only want the transcribed text to be the output.

After Idefics has processed every image patch, it inserts “\n” to signify a processed patch. These newlines have been removed in the post-OCR processing for Idefics, where both training data and test data have been stripped entirely of newlines. This results in that the post-OCR model trained on Idefics, has a different CER and WER performance, both before employing post-OCR and after employing post-OCR. In such cases, Idefics is denoted as Idefics*.

Kraken

For Kraken, where no Swedish pre-trained model yet exists, two models were selected that have been trained on a similar alphabet as Swedish. These models are the German Print Model and the Austrian Newspaper Fraktur Model. The Austrian alphabet consists of the same alphabet as the German alphabet, which incorporates all letters of the Swedish alphabet, with the exception of the letter “å” which is not present in the German alphabet. The German Print Model is a model trained for recognizing printed German and Latin texts. The model is trained on 15th century to 20th century prints. It includes 220+ characters covering Latin, special characters, fractions, superscripts, mathematical symbols, and various punctuation marks. The model was created by Stefan Weil, Jan Kamlah and Thomas Schmidt. It was released in 2023 [28].

The Austrian Newspaper Model is specifically trained for the 19th century German Fraktur texts, trained with data from Austrian Newspapers. It supports Latin script and includes 100+ characters covering Latin, numerical symbols, fractions, and various punctuation marks optimized for Fraktur typography [28].

To see the full list of pre-trained models in Kraken, see Appendix 11.

DocTR

For this thesis, DocTR has been configured to use DBNet for text detection. The hypothesis is that the flexible binarization method employed in DBNet will perform well on the source material, which may include variations in lighting and contrast. In such cases, a fixed threshold might not detect text areas as effectively. For text recognition, DocTR has been configured with a

CRNN model using a VGG16 backbone.

Tesseract

For investigating optimal parameters for Tesseract, there are multiple parameters that can be configured. First, Tesseract has a trained model configuration which is either `tessdata_fast` or `tessdata_best`. The `tessdata_best` is the most accurate LSTM models for Tesseract that prioritize recognition accuracy over processing speed and memory usage. These models use floating-point calculations that result in higher accuracy but require more computational resources. The `tessdata_fast` configuration is an integerized version of the LSTM models that have been optimized for speed and have been trained using a smaller neural network architecture [29].

Furthermore, Tesseract has both a specification for the specific OCR Engine and a specification for the Page Segmentation. The OCR Engines available for Tesseract version 5 are either to use the LSTM/neural network (`-- oem 1`) or use the Legacy Tesseract (`-- oem 0`). As this thesis aims to explore newer models and not use legacy, the LSTM/neural network option was selected as configuration for the OCR Engine. The Page Segmentation Mode (psm) for version 5 is either Fully automatic page segmentation (`psm 3`) or to assume a single uniform block of text (`psm 6`) [30]. As newspaper paragraphs are generally single uniform blocks of text, both of these page segmentation modes were assessed to obtain optimal configurations for Tesseract given the source material. These configurations were tested on 10% of the source material and the results of exploring the optimal parameters are shown in Table 2.

Table 2: OCR Performance Metrics Across Different Time Periods for Tesseract Models

Period	Model	CER	WER	count
1818-1859	Tesseract_v5.5.0_oem1_psm3	0.133	0.469	251
	Tesseract_v5.5.0_oem1_psm6	0.133	0.467	257
1860-1899	Tesseract_v5.5.0_oem1_psm3	0.106	0.320	1205
	Tesseract_v5.5.0_oem1_psm6	0.102	0.315	1242
1900-1939	Tesseract_v5.5.0_oem1_psm3	0.040	0.157	1088
	Tesseract_v5.5.0_oem1_psm6	0.041	0.161	1144
1940-1979	Tesseract_v5.5.0_oem1_psm3	0.055	0.143	898
	Tesseract_v5.5.0_oem1_psm6	0.058	0.150	939
1980-2018	Tesseract_v5.5.0_oem1_psm3	0.017	0.069	515
	Tesseract_v5.5.0_oem1_psm6	0.017	0.074	536

From Table 2 the model parameters with `oem1` and `psm3` performs best for three of the time periods, however the model parameters with `oem1` and `psm6` performs best for two of the time periods and also consistently successfully processes more files. Here, the page configuration with automatic page segmentation (`psm3`) may fail when processing certain files, where assuming a uniform block of text succeeds. As the model configuration with `oem1` and `psm6` resulted in near-comparable performance while successfully processing more files, it is considered the best model configuration.

7.4 Synthetic data generation

The synthetic data generation consists of two parts:

- Generate synthetic data on material from the newspaper paragraphs
- Generate synthetic data on material from Wikipedia

The purpose of the synthetic data is to generate additional images for model training. This includes creating data with variations in typeface, background, blur, distortion, and stroke width. The synthetic data was generated using the TRDG (Text Recognition Data Generator) package.²

²The TRDG package is available on GitHub: [TextRecognitionDataGenerator](https://github.com/ericniebler/textrecognitiondatagenerator)

In this section, first the different options to generate synthetic data will be explained and motivated, before explaining how the synthetic data was generated for the newspaper paragraphs and Wikipedia material respectively.

7.4.1 Synthetic Data Generation Parameters

The parameters used to generate the synthetic images are which font to use, the blur amount, text distortion, background, and the stroke width of the text. When the font is selected, only one font is used for generating the entire image. Furthermore, the background is always set to *Gaussian noise*. All other parameters can have a range of values and it is not mutually exclusive which setting is active. That is, an image could have blur, text distortion, and a heavy stroke width.

Typefaces

In order to generate synthetic images that are most similar to the source material, I searched and assessed free-to-use typefaces and compared these to samples from the source material. Furthermore, as the source material consists of Swedish and also partially Fraktur typeface, I evaluated and found typefaces that not only look most similar to the samples but also has support for the different symbols used in the source material, such as [å, ä, ö, Å, Ä, Ö, ƒ].

Antiqua

For the Antiqua typeface, the specific typeface was selected that was most similar to the typeface displayed in the sample seen in Figure 31.

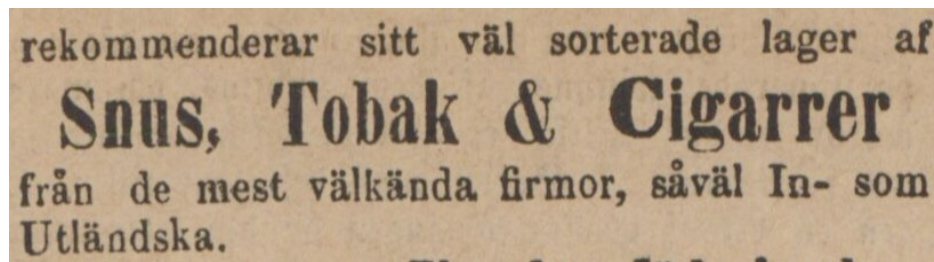


Figure 31: Antiqua font sample

With the ground truth text:

```
< aq >rekommenderar sitt väl sorterade lager af  
< b > Snus, Tobak & Cigarrer < /b >  
från de mest välkända firmor, såväl In- som  
Utländska.< /aq >
```

The typeface most similar to the sample was the typeface *Source Serif 4* from Google Fonts. The similarity with the first line of the Antiqua sample image is seen in Figure 32.

Regular 400

rekommenderar sitt väl sorterade lager af

Figure 32: Antiqua font sample, Google Font Source Serif 4

Fraktur

For Fraktur, the typeface was selected that was most similar to the typeface displayed in Figure 33.

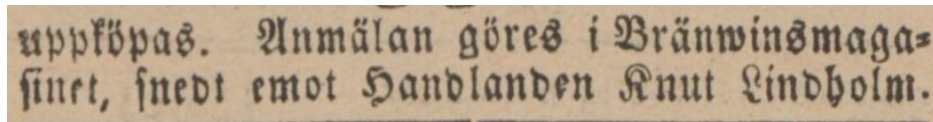


Figure 33: Fraktur font sample

With the ground truth text:

```
< fr >uppköpas. Anmälan göres i Bränwinsmaga=  
finet, fnetd emot Handlanden Knut Lindholm.< /fr >
```

Note that the “k” from the ground truth in the first word is looking similar to an “f” or a “t” in the Fraktur sample seen in Figure 33. Also note that Fraktur has the long-s *ſ*, which looks like an “f” but the horizontal line is cut short on the right side at the intercept.

The font *Wieynk Fraktur Font Family* was chosen from 1001 Fonts as it looked similar to the sample image, as seen in Figure 34.

uppköpas. Anmälan göres i Bränwinsmaga= finet, fnetd emot Handlanden Knut Lindholm.

Figure 34: Fraktur font sample, Wieynk Fraktur Font Family

Schwabacher

For Schwabacher, the typeface most similar to Figure 35 was selected.

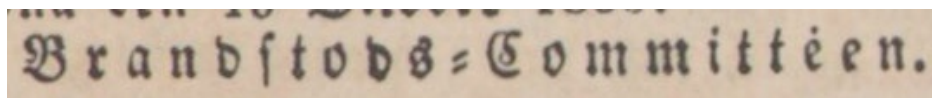


Figure 35: Swabacher font sample

With the ground truth text:

```
< sw >Brandstods=Committéen.< /sw >
```

The font *OffenbacherSchwabacherCAT Font* from 1001 Fonts was selected as it looked similar, as seen in Figure 36.

Brandstods=Committéen.

Figure 36: Swabacher font sample, OffenbacherSchwabacherCAT Font

No typeface

For when no typeface is annotated, the typeface most similar to the sample seen Figure 37 was selected.

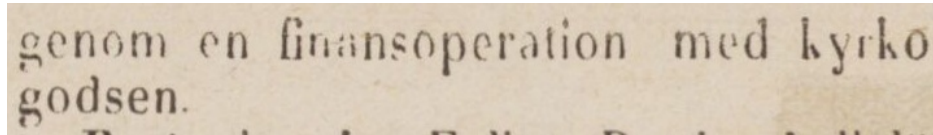


Figure 37: Regular font sample

With the ground truth text:

genom en finansoperation med kyrko godsen

The typeface *PT Serif* from Google Fonts was chosen as it looked similar, as seen in Figure 38.

genom en finansoperation med kyrko godsen.

Figure 38: Regular font, PT Serif

Blur

As text in old documents sometimes can be blurry, to modify the different blur levels and see how that training data may influence OCR is interesting. In the TRDG package, blur amount can be set with an integer value, I have selected the blur values I believe are most interesting to test, ranging from no blur (blur value 0) to maximum blur (blur value 5). If even more blur is used, then the text becomes completely unreadable. The different blur values are visible in Figure 39.

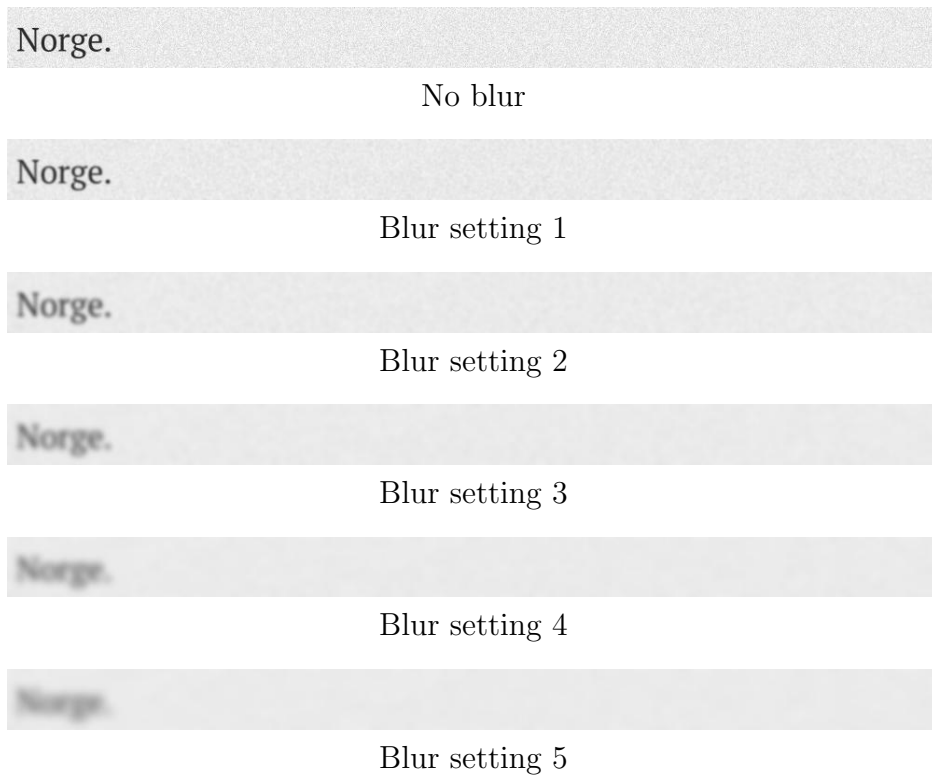


Figure 39: Comparison of different blur settings

Background

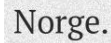
One could have multiple different backgrounds, but for the sake of OCR I believe only *Gaussian noise* is a fitting Background. One could have used a plain white background, but as the source material has various quality, and never is as “clear” as a plain white background, that option was discarded. Furthermore, the Gaussian noise background not only is most similar to the source material but the choice is also motivated by the fact that it is generated through tiny randomly placed pixels, which might affect the training in interesting ways.



Figure 40: Gaussian noise background

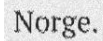
Text distortion

As real documents may have distorted text due to physical damage, print variations or perspective issue from the camera captures, it is interesting to use different text distortions for the synthetic data. The different text distortion settings are visible in Figure 41.



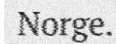
Norge.

No text distortion



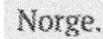
Norge.

Text distortion setting 0



Norge.

Text distortion setting 1



Norge.

Text distortion setting 2

Figure 41: Comparison of different text distortion settings

Stroke Width Modifications

As the source material consists of printed text, and historical and printed material may have inkbleed, it is of interest to use different stroke widths for the generated text. For the different stroke widths, I selected the values 0, 1, 3, 5 and 7. These are visible in Figure 42.

The word "Norge." is rendered in a standard serif font with a regular weight and no stroke width.

No stroke width

The word "Norge." is rendered in a standard serif font with a regular weight and a stroke width setting of 1.

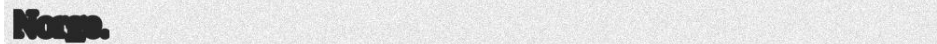
Stroke width setting 1

The word "Norge." is rendered in a standard serif font with a regular weight and a stroke width setting of 3.

Stroke width setting 3

The word "Norge." is rendered in a standard serif font with a regular weight and a stroke width setting of 5.

Stroke width setting 5

The word "Norge." is rendered in a standard serif font with a regular weight and a stroke width setting of 7.

Stroke width setting 7

Figure 42: Comparison of different stroke width settings

7.4.2 Generating the synthetic data

For generating the synthetic data, as the TRDG package does not support multi-line input, if the text input consists of more than one line, it needs to be parsed line by line. This means, that for each line, a single synthetic image is generated. After all lines have been processed and a synthetic image has been generated for each line respectively, these synthetic images are then assembled together into a multi-line image. As the lines in a paragraph can have different lengths, the width of the output image is determined by the longest text line. This ensures that the line by line images have similar widths.

Furthermore, the settings such as blur, text distortion and stroke width have been assigned randomly to each input file while following a bell curve distribution. This can be seen in Figure 43 and Figure 44. To generate values that follow a bell curve, a float value was randomly assigned for each characteristic, which is then mapped to its corresponding integer value which is the input for the TRDG synthetic data generator.

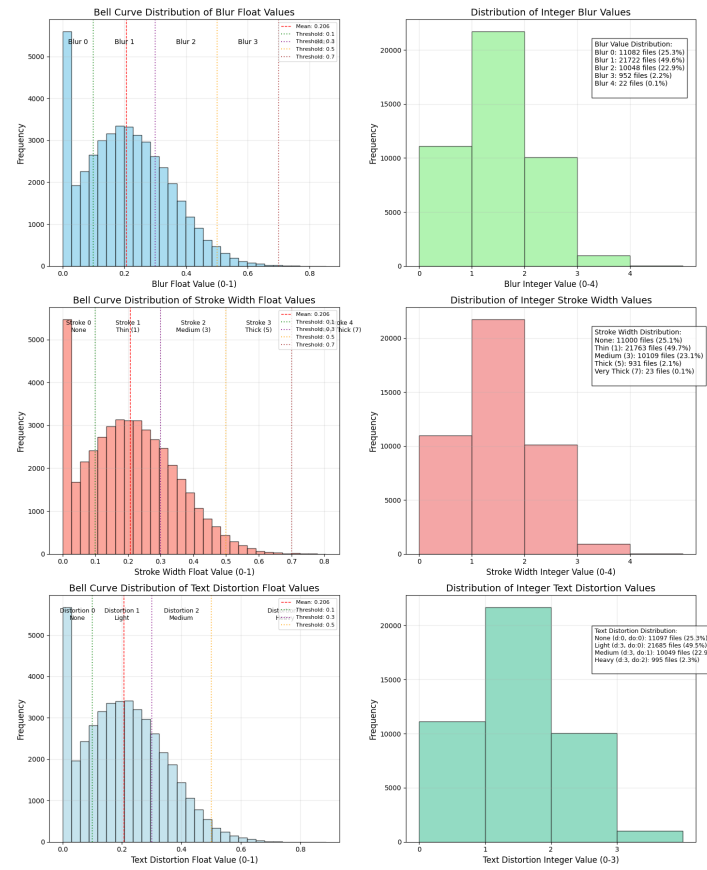


Figure 43: Distribution of blur, stroke width and text distortion for newspaper data

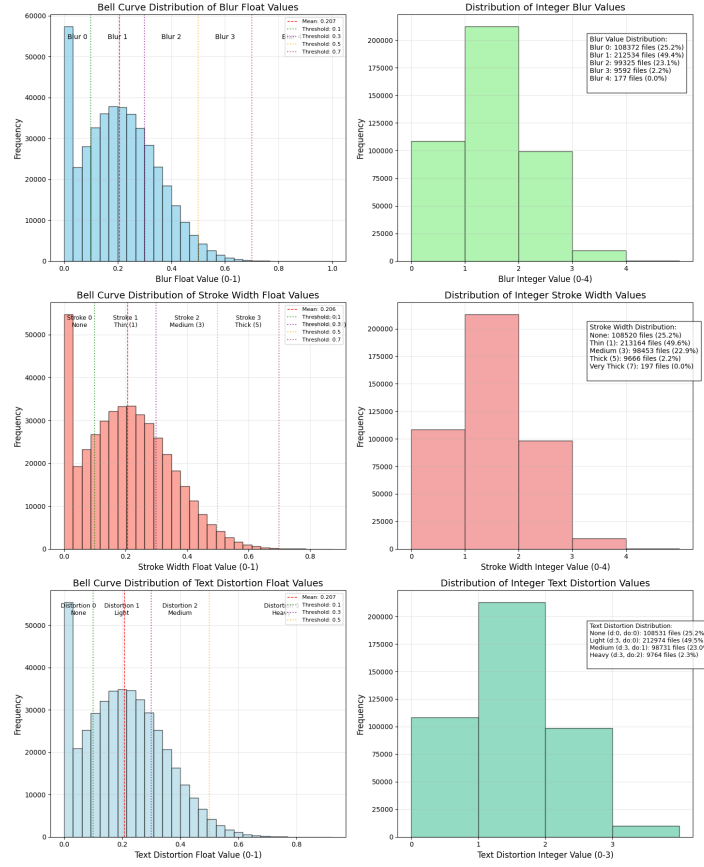


Figure 44: Distribution of blur, stroke width and text distortion for Wikipedia data

For generating synthetic images for the newspaper material, the typeface distribution from the ground truth has been used to generate a synthetic dataset that follows the same distribution. For the text contents of the synthetic images, the ground truth files from the newspaper dataset were used.

However, for the material from Wikipedia, the typeface was assigned randomly. The typeface distribution from the source material was not considered as I believe it is best to generate as much synthetic data as possible for each typeface and therefore a uniform distribution is desirable. The dataset used for generating synthetic images based of Wikipedia text is the Swedish Wikipedia dataset from Huggingface. As the dataset consists of five files

with each around 500 000 articles of various length, it was deemed sufficient to download one of these files (the first one) and use the contents of that for the synthetic data generation.

Furthermore, as Wikipedia articles can be quite long, it is important to cut the content into smaller pieces. To do this, I calculated the text length percentiles from the ground truth and defined the maximum text length to be at the 90th percentile (which is 404 characters). The Wikipedia article contents were then split into sentences with a transformer based Sentence Splitter.³ The lines from the Sentence Splitter were then added as input, while ensuring the length of each paragraph was no longer than the 90th percentile. If so, the contents were split into chunks.

7.5 Post-OCR training

For research question three, one post-OCR correction model was trained for each OCR model's output. These models are then tested across different OCR systems to assess transferability. The post-OCR model has then been evaluated on OCR output from each OCR model, in order to assess the post-OCR model's ability to correct the OCR errors present in each output.

For research question three, to explore using synthetic data in training an OCR-model, three different post-OCR models were trained. One post-OCR model was trained on synthetic newspaper data, using the ground truth from the source material. This is denoted as S^{news} . To provide more synthetic text samples, a model trained on Swedish Wikipedia texts was also developed, denoted S^{wiki} . The third post-OCR model was trained on the original source material along with the synthetic Wikipedia data, and is denoted $S^{original+wiki}$. The size of S^{news} is 43 826 images and S^{wiki} is 430 000 images (approximately 10x). These images were processed with Tesseract version 5.5.0 as it is quick to process.

The reasoning behind the training structure is to assess if and what synthetic data improves the performance of the post-OCR correction model.

³The sentence splitter is available on GitHub: [sentence.splitter](#)

7.5.1 Post-OCR correction model

For post-OCR correction, a model developed by KBLab was employed, which is a reimplementation of Löfgren and Dannélls ByT5 post-OCR model [6]. The model utilizes sequence-to-sequence transformer models (specifically ByT5) to “translate” erroneous OCR text into corrected text, treating post-OCR correction as a machine translation problem [6].

The system processes data from the specified OCR Engines with corresponding ground truth transcriptions. The data is loaded from JSON and split into training (70%), validation (15%), and test (15%) sets. To ensure fair comparison across OCR models, the same holdout files are specified and used for all engines [31]. Furthermore, the text is segmented into chunks with a maximum length of 128 characters, and samples are filtered based on quality criteria (minimum 4 characters, maximum 50% CER). This filtering ensures the model trains on files meeting minimum OCR quality [6, 31].

The model employs a pre-trained ByT5 transformer, which operates at the byte level, but for understanding purposes, this can be approximated to operating at the character level.⁴ This approach is crucial for Swedish historical newspapers as it preserves the words’ historical spelling variations (e.g., “hvad” vs “vad”) that would be lost with word-level tokenization.

Performance is measured using CER and WER. Previous research by Löfgren and Dannélls reported a 36% reduction in CER using the ByT5 post-OCR correction model, demonstrating the effectiveness of the approach [6].

7.6 Technical infrastructure

The OCR engines (Tesseract, Kraken, Idefics, and DocTR) were executed on NVIDIA DGX systems. This was chosen as infrastructure as both Idefics and DocTR needed the processing power. For example, Idefics, with 8 billion parameters was too large to train on a regular GPU supported computer. I deliberately chose to not use a quantized versions of Idefics, with the arguments to maximize precision and accuracy to ensure a fair comparison in terms of CER and WER results. Furthermore, the training of the OCR models was also done with NVIDIA DGX:es as hardware. For all other scripts,

⁴Some characters do however contain more bytes than others.

the hardware was a NVIDIA GeForce RTX 2080 Ti, with a single GPU.

To ensure reproducible environments, I containerized most scripts using Docker. The majority of the scripts also support direct execution via command-line parameters for flexibility.

7.7 Evaluation methods

The evaluation methods of CER and WER as primary evaluation metrics are motivated by the fact that CER and WER are common metrics when assessing and reporting on OCR results. CER and WER give insight into how many characters are wrongly identified and, in turn, how many words are wrongly identified. To get insight into which characters often result in a faulty recognition, confusion matrices are used to accompany the CER and WER results and give insight into which true and predicted characters are often wrongly substituted.

8 Results

This chapter presents the findings from evaluating recent OCR models on historical Swedish newspapers and exploring post-OCR correction techniques. The results are organized to address each research question systematically. First, the comparative performance of recent OCR models is analyzed to determine which systems perform best on historical material. Next, the transferability of post-OCR correction models across different OCR systems is examined. Finally, the effectiveness of synthetic data in training post-OCR models is evaluated.

8.1 Comparative performance of recent OCR models

To answer research question 1,

What is the comparative performance enhancement offered by recent OCR models?

The following results were obtained, visible in Figure 45. Notably, Idefics achieves a WER of 106%, indicating extensive insertion errors that exceed the ground truth text length. Detailed results are provided in Table 3.

Overall CER and WER Performance

All time periods, with tesseract v4.1.3 as baseline

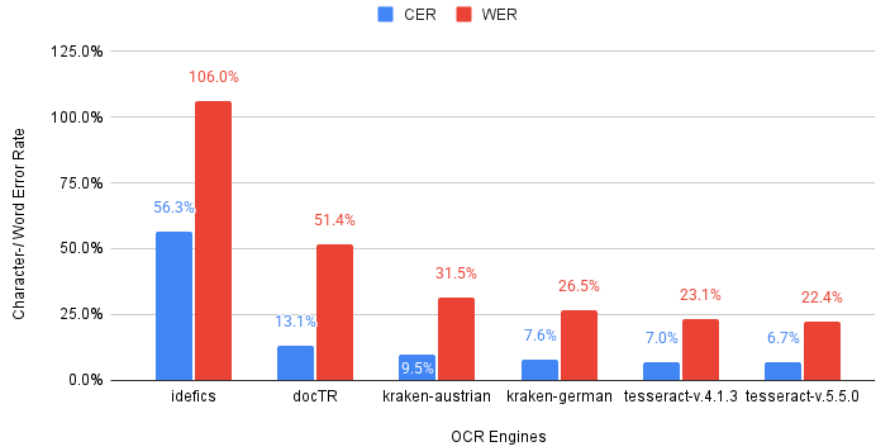


Figure 45: Overall OCR Performance per OCR Engine

Table 3: Overall OCR Performance Metrics

Model	CER	WER	count
Idefics	0.563	1.060	41664
DocTR	0.131	0.514	39218
Kraken Austrian	0.095	0.315	39364
Kraken German	0.076	0.265	39288
Tesseract_v.5.5.0	0.067	0.224	41170
Tesseract_v.4.1.3	0.070	0.231	41501

The results show that Tesseract version 5.5.0 is the OCR system that performs best in terms of Character Error Rate (CER) and Word Error Rate (WER). It is interesting to note, however, that Kraken German is a close second, with a CER of 7.6% and a WER of 26.5%. This in comparison to Tesseract version 5.5.0 that yielded CER of 6.7% and WER of 22.4%. Another finding is that the new Tesseract performs marginally better than its predecessor, with 0.3% lower CER and 0.7% lower WER results.

If we examine, however, the results for each time period, and start with

the oldest material, we see that even though Kraken German is specialized on historical material, Tesseract still performs best (in terms of CER) on the oldest material, as seen in Table 4. If we however compare the WER results, Kraken German performs better with a WER of 43% whereas Tesseract yields 49%. This is an interesting find, which signifies that Kraken German likely gets a sequence of characters wrong in the same word, rather than errors in multiple words.

Table 4: OCR Performance Metrics for Period 1818-1859

Model	CER	WER	count
Idefics	1.154	1.634	2536
DocTR	0.261	0.750	2480
Kraken Austrian	0.209	0.528	2499
Kraken German	0.152	0.430	2489
Tesseract_v.5.5.0	0.146	0.492	2514
Tesseract_v.4.1.3	0.147	0.496	2540

Similarly, if we examine the results obtained from material from the time period 1860 until 1899 Kraken German excels over the new version of Tesseract, both in terms of CER and WER, with a CER of 9% and a WER of 29%. This is seen in Table 5. DocTR performs second to last, with 16.7% CER and Idefics yields a 73% CER.

Table 5: OCR Performance Metrics for Period 1860-1899

Model	CER	WER	count
Idefics	0.729	1.215	12668
DocTR	0.167	0.578	12091
Kraken Austrian	0.112	0.346	11839
Kraken German	0.090	0.289	11788
Tesseract_v.5.5.0	0.095	0.300	12461
Tesseract_v.4.1.3	0.098	0.306	12635

If we examine the results from the first part of the 20th century, we see in Table 6 that Tesseract version 5.5.0 now performs with a CER of 3.9% and a WER of 16%. The improvement in CER and WER for Tesseract implies that it has most likely been trained on modern material rather than material from

the 1800s. The same goes for DocTR and Idefics, which yielded the highest CER and WER scores on older material, and have thus greatly improved.

Table 6: OCR Performance Metrics for Period 1900-1939

Model	CER	WER	count
Idefics	0.443	0.924	11586
DocTR	0.089	0.443	10846
Kraken Austrian	0.063	0.264	10939
Kraken German	0.052	0.229	10936
Tesseract_v.5.5.0	0.039	0.161	11494
Tesseract_v.4.1.3	0.040	0.164	11556

The results obtained through material from 1940-1979 is presented in Table 7. Again, version 5.5.0 of Tesseract performs better than the other OCR engines, both in terms of CER and WER.

Table 7: OCR Performance Metrics for Period 1940-1979

Model	CER	WER	count
Idefics	0.401	0.915	9699
DocTR	0.105	0.447	8915
Kraken Austrian	0.083	0.254	9147
Kraken German	0.069	0.220	9137
Tesseract_v.5.5.0	0.053	0.145	9571
Tesseract_v.4.1.3	0.059	0.158	9626

For the last time period, Tesseract version 5.5.0 performed best, with a staggering result of 1.9% CER and 7.5% WER. This is visible in Table 8. For this time period, no other model is close to those results, as Kraken German on the second place yields 4.6% in CER.

Table 8: OCR Performance Metrics for Period 1980-2018

Model	CER	WER	count
Idefics	0.228	0.727	5175
DocTR	0.076	0.421	4886
Kraken Austrian	0.056	0.267	4940
Kraken German	0.046	0.223	4938
Tesseract_v.5.5.0	0.019	0.075	5130
Tesseract_v.4.1.3	0.021	0.080	5144

8.1.1 Error analysis

Substitution errors

As CER consists of three types of errors; substitution errors, insertion errors and deletion errors, substitution errors are most interesting to assess as they give insight into which character was wrongly recognized as another character. This yields insight into the strengths and weaknesses of each model. The confusion matrices are added in the Appendix 11, and a brief summary of each OCR engines top three substitutions are noted and assessed here.

For Tesseract version 5.5.0, the top three errors are the fraktur long-s, that have wrongly been identified as an “f” (in 21 738 instances), followed by “k” being identified as “f” (in 7 187 instances). Recall from the method section that the Fraktur long-s “ſ” and the Fraktur “k” are both very similar to an “f” in shape, as explained in the Method section. The third most common error is a period, that is being recognized as a comma (4 802 instances).

For Kraken German, the top three errors are “ä” being identified as “ä” (33 385 instances), “ä” being identified as “à” (22 202 instances) and the letter “ä” being identified as an “a” (9 429 instances). Kraken German does not have any problems in identifying the Fraktur “long-s” “ſ”.

For Kraken Austrian, the top three errors are “ä” being identified as “ä” (59 360 instances), the character “=” being identified as a double-oblique hyphen (5 010 instances) and comma being identified as a period (3 986 instances).

For DocTR, the top three errors are “ö” being identified as “o” (66 774

instances), “ä” being identified as “a” (43 676 instances) and the letter “å” being identified as “à” (27 204 instances). Thus, it clearly has problems with all the special Swedish characters. Similar top three errors are found in Idefics, where the top three errors are “å” being identified as “a” (28 599 instances), “ä” being identified as “a” (26 228 instances) and “ö” being identified as “o” (22 216 instances).

Therefore, I can conclude that the Swedish letters are predominantly the letters that are most substituted for the majority of the OCR engines. However, Tesseract, which had a Swedish language setting, did not have particular problems with the letters. Both Kraken models and DocTR are not trained on Swedish, however Idefics did get a prompt which specified the Swedish language.

Examples that yielded worst CER/WER

As some models, like Idefics and DocTR, yielded the highest result in CER and WER, it is interesting to assess the results from each model that yielded the worst CER.

For Idefics, we see that the results with the worst CER are files where Idefics continued to generate words, long after the text was OCR:ed. One example is an image that only contained the text “Barn och Barnbarn” where Idefics generated the word “Barn” 1500 times. Of the 10 results with the worst CER, 8 of these implies that Idefics got stuck in a loop and continued to generate the same sequence over and over.

However, it should be noted that for all OCR engines, the worst CER results were always given where text on the images was not properly transcribed, there are some instances where the transcription simply is “*”, “D”, “ASEA” or else where the image actually contains more text which is not found in the transcription. But as all OCR engines therefore yields a high CER and WER for these, the inherent differences between the models performance are still the same.

One example is shown in Figure 46, where the transcription reads “ASEA” but the image clearly contains more text.



Figure 46: Example of inaccurately transcribed text

Another example is Figure 47 where the initials in the right corner are transcribed. In this instance, Tesseract produced gibberish with a CER of 580%.



Figure 47: Example of image paragraph with initials

Therefore, one additional filtering has been made to show comparable OCR performance. Images which yield high CER (over 50%) for any of the OCR engines, are filtered out. With the filtering made, only files with CER lower than 50% are kept to assess CER and WER for each OCR Engine. The results from the kept 15 935 files are shown in Table 9. Tesseract version 4.1.3 yielded the best results in terms of CER, but Tesseract version 5.5.0 yielded the best results in terms of WER. However, when assessing performance rounded to three decimal places, the results are comparable with no significant difference.

Table 9: Overall OCR Performance - CER - Filtered files

Model	CER	CER filtered	CER Diff.
Idefics	0.563	0.188	-67%
DocTR	0.131	0.111	-15%
Kraken Austrian	0.095	0.077	-19%
Kraken German	0.076	0.065	-14%
Tesseract_v.5.5.0	0.067	0.056	-16%
Tesseract_v.4.1.3	0.070	0.056	-20%

Table 10: Overall OCR Performance - WER - Filtered files

Model	WER	WER filtered	WER Diff.
Idefics	1.060	0.660	-38%
DocTR	0.514	0.511	-1%
Kraken Austrian	0.315	0.370	+17%
Kraken German	0.265	0.334	+26%
Tesseract_v.5.5.0	0.224	0.289	+29%
Tesseract_v.4.1.3	0.231	0.289	+25%

8.2 Post-OCR correction model transferability

The results when performing post-OCR on OCR-output from each model are found within Tables 11-14. *Note:* As Idefics inserts newlines (specifically “\n”) after every processed image patch, Idefics results differ as the entire training and testing procedure have been stripped of “\n”. These entries have been marked with “*”.

For OCR output generated by Tesseract, it yielded the best post-OCR improvement when corrected by a post-OCR correction model trained specifically on Tesseract, with a 13% improvement in CER and approximately 29% improvement in WER. This can be seen in Table 11. Furthermore, DocTR and Idefics also improved Tesseract, but only in terms of WER. In terms of CER, Idefics negatively affected the OCR output. The reason WER improves while CER worsens is likely that more words overall are being corrected, while those words that remain incorrect undergo multiple character changes, deletions, or insertions, which worsens CER. In short: more words

in a sentence are corrected, while those words that are still wrong become even more misspelled after post-OCR correction.

Table 11: Post-OCR correction model performance on Tesseract output with CER of 0.128 and WER of 0.270. Idefics* had CER 0.086 and WER 0.228 before post-OCR.

P.OCR Model	CER	Diff.	WER	Diff.
Tesseract	0.111	-13.0%	0.193	-28.5%
DocTR	0.128	0%	0.245	-9.5%
K.German	0.134	+5.3%	0.266	-1.5%
Idefics*	0.098	+14.7%	0.206	-9.6%

The results of correcting OCR output from DocTR with different post-OCR models are particularly noteworthy. DocTR’s initial output yielded a CER of 15.4%, which when corrected by a post-OCR model trained specifically on DocTR data improved to 9.9% - representing approximately a 36% improvement in CER. Similarly, at the word level, it achieved a 58% improvement in WER. This finding is significant as it demonstrates that even suboptimal initial OCR results can be corrected to achieve performance that surpasses both the best-performing baseline OCR model (Tesseract version 5.5.0 with 12.8% CER) and its post-OCR corrected output (11.1% CER).

OCR output from DocTR was also substantially improved when processed by a post-OCR model trained on Idefics data, achieving approximately 27% improvement in CER and 49% improvement in WER. These cross-model improvements suggest that DocTR output likely follows certain error patterns that are also present in other OCR systems’ outputs, allowing post-OCR models trained on different data to still provide meaningful corrections. The results are shown in Table 12.

Table 12: Post-OCR correction model performance on DocTR output with CER of 0.154 and WER of 0.509. Idefics* had CER 0.171 and WER 0.554 before post-OCR.

P.OCR Model	CER	Diff.	WER	Diff.
DocTR	0.099	-35.5%	0.212	-58.3%
Tesseract	0.136	-11.9%	0.399	-21.7%
K.German	0.160	+3.7%	0.389	-23.6%
Idefics*	0.125	-26.7%	0.284	-48.9%

For OCR output from Idefics, the best improvement in performance was achieved with a post-OCR model trained on Idefics data, resulting in a 57% improvement in CER and a 66% improvement in WER. This can be found in Table 13. This substantial improvement suggests that Idefics produces consistent, learnable error patterns that can be effectively corrected by a matching post-OCR model.

However, post-OCR models trained on other systems (DocTR, Tesseract, or Kraken German) provided only marginal improvements to Idefics output, with CER improvements ranging from 1.2% to 3.4%. This limited effectiveness is likely due to a significant methodological difference: the Idefics post-OCR model was trained on data where newline characters (`\n`) had been entirely stripped from both the OCR output and training data, while the other post-OCR models were trained on data containing these characters. When applied to standard Idefics output (which contains `\n` characters), these models likely struggle with this formatting inconsistency, limiting their correction capability. Furthermore, it could also be difficult for these post-OCR models to correct the underlying OCR data, as there might exist hallucinations or similar.

Table 13: Post-OCR correction model performance on Idefics output with CER of 1.582 and WER of 1.934. Idefics* had CER 0.130 and WER 0.194 before post-OCR.

P.OCR Model	CER	Diff.	WER	Diff.
Idefics*	0.130	-56.5%	0.194	-65.5%
DocTR	1.528	-3.4%	1.813	-6.3%
Tesseract	1.563	-1.2%	1.891	-2.2%
K.German	1.558	-1.5%	1.868	-3.4%

Output from Kraken German was most improved by a post-OCR model trained on Kraken German, resulting in a 8.5% improvement in CER and a remarkable 34% improvement in WER. This is visible in Table 14. This suggests that Kraken German produces consistent error patterns that can be learned and corrected by its own post-OCR correction model.

However, the other post-OCR models show minimal or negative impact on Kraken German output on a character level. This suggests that Kraken German’s error patterns are distinct from other OCR systems, making cross-model correction less effective in post-OCR correction. On a word level, these models improve WER to some degree (ranging from 12.6% to 26.0%). This is likely due to that more words are being corrected by these models overall, but character errors still occur within the words that are wrongly recognized.

Table 14: Post-OCR correction model performance on Kraken German output with CER of 0.117 and WER of 0.299. Idefics had CER 0.095 and WER 0.286 before post-OCR.

P.OCR Model	CER	Diff.	WER	Diff.
K.German	0.107	-8.5%	0.197	-34.0%
DocTR	0.119	+2.0%	0.244	-18.2%
Tesseract	0.115	-2.0%	0.261	-12.6%
Idefics*	0.097	+2.0%	0.211	-26.0%

8.2.1 Error analysis after post-OCR

If one analyzes what drives the CER improvements and assesses the underlying insertion, deletion, and substitution error patterns corrected by the post-

OCR model, one can determine the specific correction patterns the model has learned and employs. The results for each best-performing post-OCR model are presented below.

Tesseract

For Tesseract the post-OCR model trained on Tesseract significantly decreases the number of insertion- and substitution errors. This is visible in Table 15.

Table 15: Post-OCR Model’s correction on Tesseract - Error type summary

Error Type	OCR Count	P.OCR Count	% Change
Insertions	9,850	6,495	-34.1
Deletions	23,241	23,438	+0.8
Substitutions	30,207	20,578	-31.9

The top ten OCR Errors within the Tesseract OCR output, and the post-OCR models affect on these are visible in Table 16. The results show a -95% change in the substitution error *ſ* to “f” that was prominent in Tesseracts OCR output. Similarly the common substitution of “k” to “f” which Tesseract produced have been decreased significantly. These results demonstrate that the post-OCR model successfully learned to address Tesseract’s systematic errors, particularly its tendency to misinterpret historical typographic elements (such as *ſ*→f) and common character confusions (k→f, w→m).

Table 16: Post-OCR Model’s correction on Tesseract - Top character changes by absolute change

Character(s)	Error Type	OCR Count	Change	% Change
ſ → f	Substitution	3,000	-2,851	-95.0
k → f	Substitution	1,019	-963	-94.5
\n	Deletion	13,554	+730	+5.4
period → comma	Substitution	690	-464	-67.2
ſk → ff	Substitution	435	-435	-100.0
k	Deletion	502	-348	-69.3
w → m	Substitution	346	-302	-87.3
\t → space	Substitution	502	-289	-57.6
c → d	Substitution	274	-242	-88.3
= → s	Substitution	226	-226	-100.0

DocTR

DocTR performed exceptionally well with the post-OCR model trained on DocTR. When one assesses the underlying reasons for this, we can conclude that the DocTR post-OCR model learned to correct substitution errors. This is visible in Table 17.

Table 17: Post-OCR model’s correction on DocTR - Error type summary

Error Type	OCR Count	P.OCR Count	% Change
Insertions	8,728	6,752	-22.6
Deletions	26,258	23,748	-9.6
Substitutions	75,103	21,649	-71.2

If one assesses which characters substitution errors which were corrected, we can see that it clearly is the Swedish special characters (“å, ä, ö”) and ſ to “f” that have been corrected. This is visible in Table 18. This signifies that even though an OCR model is not trained on Swedish language, it can to a great extent be corrected by a post-OCR correction step, as long as the patterns are easily learnable.

Table 18: Post-OCR model’s correction on DocTR - Top character changes by absolute change

Character(s)	Error Type	OCR Count	Change	% Change
ö → o	Substitution	9,975	-9,708	-97.3
ä → a	Substitution	6,539	-6,077	-92.9
å → à	Substitution	4,000	-4,000	-100.0
d → b	Substitution	3,899	-3,840	-98.5
å → â	Substitution	3,428	-3,428	-100.0
ä → à	Substitution	2,447	-2,447	-100.0
å → a	Substitution	2,517	-2,294	-91.1
ä → â	Substitution	2,009	-2,009	-100.0
f → f	Substitution	1,874	-1,760	-93.9
\n	Deletion	13,512	+1,259	+9.3

Idefics

Idefics also had repeatable OCR error patterns in the OCR output, which a post-OCR model trained on Idefics learned and corrected. Across all underlying categories that constitutes CER; insertion-, deletion- and substitution errors were improved by 59%-71.6%, visible in Table 19.

Table 19: Post-OCR model’s correction on Idefics - Error type summary

Error Type	OCR Count	P.OCR Count	% Change
Insertions	11,807	4,382	-62.9
Deletions	14,077	5,772	-59.0
Substitutions	30,118	8,568	-71.6

Analysis of the most dominant errors and the post-OCR model’s corrections reveals that the substantial CER reduction is primarily driven by over 5000 corrections of Swedish vowel characters (å→a, ä→a, ö→o), combined with the model’s ability to identify and correct erroneous character deletions throughout the text. This pattern is clearly visible in Table 20.

Table 20: Post-OCR model’s correction on Idefics - Top character changes by absolute change

Character(s)	Error Type	OCR Count	Change	% Change
å → a	Substitution	2,052	-1,993	-97.1
ä → a	Substitution	1,880	-1,797	-95.6
ö → o	Substitution	1,613	-1,584	-98.2
n	Deletion	701	-460	-65.6
l	Deletion	572	-394	-68.9
r	Deletion	604	-373	-61.8
t	Deletion	606	-371	-61.2
space	Deletion	439	-355	-80.9
å → ä	Substitution	349	-313	-89.7
s	Deletion	492	-296	-60.2

Lastly for Kraken German, the improvement in CER stems from a decrease in substitution errors with 41.3%, as seen in Table 21.

Table 21: Post-OCR model’s correction on Kraken German - Error type summary

Error Type	OCR Count	P.OCR Count	% Change
Insertions	5,282	6,869	+30.0
Deletions	23,764	22,732	-4.3
Substitutions	38,410	22,554	-41.3

The substitution changes made by the post-OCR model are also centered around the Swedish characters (“å, ä, ö”) which accounted for more than 9000 substitution errors, as seen in Table 22.

Table 22: Post-OCR Model’s correction on Kraken German - Top character changes by absolute change. (Double oblique hyphen is abbreviated to “d.o. hyphen”)

Character(s)	Error Type	OCR Count	Change	% Change
å → ä	Substitution	4,876	-4,651	-95.4
å → à	Substitution	3,368	-3,368	-100.0
å → a	Substitution	1,470	-1,241	-84.4
= → d.o. hyphen	Substitution	773	-773	-100.0
period	Insertion	216	+666	+308.3
k → t	Substitution	506	-397	-78.5
l → i	Substitution	387	-317	-81.9
\n	Deletion	13,741	+281	+2.0
t → i	Substitution	334	-279	-83.5
comma → period	Substitution	515	-265	-51.5

Summary

Across all OCR systems, the post-OCR models show remarkable effectiveness in correcting language-specific characters (å, ä, ö), regardless of the underlying OCR technology. This suggests that character-level correction models can successfully compensate for OCR systems’ limitations in handling Swedish text.

8.3 Training with synthetic data

The results of training three models on different synthetic data are visible in Table 23. For the post-OCR model trained on Synthetic Newspaper data, S^{news} has an increase in CER and a slight decrease in WER. This indicates that the post-OCR model is making slightly beneficial corrections at the word level despite character-level trade-offs. However for synthetic data trained on Wikipedia dataset, S^{wiki} , shows a degradation in both CER and WER, with 197% increase in CER and 126.7% increase in WER. This suggests that the model may be over-correcting or introducing systematic biases trained from the post-OCR data. Lastly for the combined synthetic post-OCR model, $S^{original+wiki}$, achieves best overall performance with a slight CER increase (+6.1%) but substantial WER improvement (-15.9%). The increase in WER performance suggests better semantic understanding from a combined approach. However, the results are still not on par with performing post-OCR

without synthetic data, which achieves better results than $S^{original+wiki}$.

Table 23: Post-OCR correction results with synthetic data. Baseline CER of 0.128 and WER of 0.270.

P.OCR Model	P.OCR	Diff.	P.OCR	Diff.
S^{news}	0.139	+9.2%	0.265	-1.9%
S^{wiki}	0.379	+197%	0.613	+126.7%
$S^{original+wiki}$	0.135	+6.1%	0.227	-15.9%

8.3.1 Error analysis after post-OCR

The error pattern analysis reveals the underlying mechanisms driving these performance differences by examining the insertion, deletion, and substitution errors made by the post-OCR model compared to ground truth. This analysis provides insight into what types of corrections the post-OCR model is attempting to make and whether these corrections are beneficial or detrimental.

It is important to note that the following error analysis examines absolute error counts, whereas CER relates total character-level errors to the total number of characters in the ground truth, and WER relates word-level errors to the total number of words in the ground truth. Therefore, post-OCR may exhibit fewer total errors while still yielding higher CER and WER scores as seen in Table 23.

Newspaper synthetic

For the synthetic newspaper data we can see that the post-OCR model decreases the number of erroneous substitutions and deletions, but introduces insertion errors. This is visible in Table 24, where the top character changes are visible in Table 25.

Table 24: Newspaper synthetic’s correction on Tesseract - Error type summary

Error Type	OCR Count	P.OCR Count	% Change
Insertions	9,850	11,739	+19.2
Deletions	23,241	22,397	-3.6
Substitutions	30,207	25,671	-15.0

If we assess the top 10 errors by the post-OCR Model we can see that the post-OCR model corrects common Fraktur errors (such as $\mathfrak{f} \rightarrow f$, $k \rightarrow f$, $\mathfrak{k} \rightarrow ff$) but that it also introduces entirely new errors (curly $\rightarrow$ straight citation marks and $k \rightarrow \mathfrak{f}$).

Table 25: Post-OCR model’s correction on newspaper synthetic - Top character changes by absolute change

Character(s)	Error Type	OCR Count	Change	% Change
$\mathfrak{f} \rightarrow f$	Substitution	3,000	-2,867	-95.6
$k \rightarrow f$	Substitution	1,019	-933	-91.6
curly citation $\rightarrow$ straight	Substitution	0	+485	NEW
period $\rightarrow$ comma	Substitution	690	-442	-64.1
$\mathfrak{k} \rightarrow ff$	Substitution	435	-435	-100.0
$k \rightarrow \mathfrak{f}$	Substitution	0	+168	NEW
$= \rightarrow s$	Substitution	226	-150	-66.4
$= \rightarrow -$	Substitution	221	+145	+65.6
$h \rightarrow b$	Substitution	225	-144	-64.0
comma	Insertion	126	+123	+97.6

Wikipedia synthetic

For the synthetic Wikipedia post-OCR model we can in Table 26 see that it introduces a steep increase in insertions (+189.9%) and substitutions (+33.9%).

Table 26: Wikipedia synthetic’s correction - Error type summary

Error Type	OCR Count	P.OCR Count	% Change
Insertions	9,850	28,551	+189.9
Deletions	23,241	19,628	-15.5
Substitutions	30,207	40,438	+33.9

Some of these substitution- and insertion errors are visible in Table 27. The post-OCR model correctly identifies $\backslash n$ and $\mathfrak{f} \rightarrow f$, but it also introduces new errors, where $\backslash n \rightarrow$ space and $\mathfrak{f} \rightarrow k$ are a few.

Table 27: Post-OCR model’s correction on Wiki Synth - Top character changes by absolute change

Character(s)	Error Type	OCR Count	Change	% Change
'\n'	Deletion	13,554	-2,618	-19.3
f → f	Substitution	3,000	-1,968	-65.6
\n → space	Substitution	175	+1,181	+674.9
f → k	Substitution	0	+1,040	NEW
k → f	Substitution	1,019	-933	-91.6
— → –	Substitution	0	+766	NEW
f → s	Substitution	0	+651	NEW
space → \t	Substitution	0	+486	NEW
's'	Insertion	147	+435	+295.9
fk → ff	Substitution	435	-372	-85.5

Original source material + Wikipedia synthetic

For the combined approach, where the post-OCR model was trained on both synthetic newspaper data and synthetic Wikipedia data, we see that the substitution errors and insertion errors have been improved by the post-OCR model. This is visible in Table 28. From the top 10 most common error changes employed by the post-OCR model, we see that it correctly handles previous Fraktur errors. This is shown in Table 29. That no insertion errors are visible in the table suggests that the insertion errors are spread out for multiple different characters, rather than centered around a particular character.

Table 28: Original + Wikipedia synthetic’s correction on Tesseract - Error type summary

Error Type	OCR Count	P.OCR Count	% Change
Insertions	9,850	7,895	-19.8
Deletions	23,241	22,865	-1.6
Substitutions	30,207	20,905	-30.8

Furthermore, the post-OCR model persists to have deletion errors, where newlines are being deleted that are in fact present in the ground truth. This is visible in Table 29.

Table 29: Post-OCR model’s correction on original+wiki synth - Top character changes by absolute change

Character(s)	Error Type	OCR Count	Change	% Change
f → f	Substitution	3,000	-2,846	-94.9
k → f	Substitution	1,019	-988	-97.0
\n	Deletion	13,554	+644	+4.8
period → comma	Substitution	690	-456	-66.1
fk → ff	Substitution	435	-435	-100.0
k	Deletion	502	-352	-70.1
w → m	Substitution	346	-301	-87.0
c → d	Substitution	274	-253	-92.3
\t → space	Substitution	502	-236	-47.0
= → s	Substitution	226	-201	-88.9

9 Discussion

This chapter analyzes and interprets the findings from the OCR evaluation and post-OCR correction experiments. The discussion examines the performance patterns observed across different OCR models and their implications for processing historical Swedish newspapers. It explores the factors that influence model performance, including typography, language support, and architectural choices. The transferability of post-OCR correction models is then discussed, followed by an analysis of synthetic data's role in training correction models.

9.1 Comparing OCR model performance

Typography- & language-specific performance patterns

While Tesseract yielded the best overall results, Kraken German did however excel with material from 1860-1899, a period in which both Fraktur and Antiqua were present in the source material. This suggests that specialized OCR models like Kraken German may offer advantages for processing specific typography styles during certain historical periods. Kraken German outperformed Tesseract in this regard, even though Kraken was not trained on the Swedish language and consistently recognized “å” as other letters.

Tesseract and Idrefics were the two models in which the language has been specified, which resulted in Tesseract's dominant errors to be ones related to Fraktur-typeface. Idrefics however, even though the language was specified in the prompt, still struggled with Swedish characters å, ä, ö. This highlights that vision language models need to be fine-tuned for each language.

These language-specific and typography-specific patterns demonstrate the dual challenge of performing OCR on historical material, as both a language-specific character recognition is needed as well as a typography-specific recognition, in order to yield the best possible results when doing OCR on historical material.

Historical vs modern material performance

The contrast in performance between recent source material (the period 1980-2018) and older source material (1818-1859) across all OCR models indicates a persistent challenge in digitizing historical documents - even with recent

OCR models. Even the best-performing OCR model, Tesseract version 5.5.0, showed approximately an eight-fold increase in CER when comparing older source material against the most recent source material. These performance discrepancies signify that recent OCR models are still optimized for modern source material, spelling and typography. For institutions digitizing historical and modern material, this indicates that modern material can be processed with high confidence whereas historical material needs more specialized approaches both in OCR and/or the post-OCR process.

Impact of ground truth quality

It should be noted that there were instances in the source material where the transcription was not complete, i.e. where more words were present in the paragraph than what was actually transcribed in ground truth. This affects OCR model performance, as all OCR models get abnormally high CER and WER results, driving up the overall performance metrics.

In order to assess true OCR performance, one attempt was made to filter out the instances that yielded high CER results for any of the OCR models. This filtering ensures that all files were present in the performance metrics of every OCR model, offering a comparable performance on the files that no OCR model had any notable difficulty with. This resulted in an increase in CER performance visible in Table 9. However, the filtering negatively impacted WER as seen in Table 10.

The increase in CER performance is likely due to the OCR model being penalized for transcribing text in the images that was not included in the incomplete ground truth files, resulting in artificially inflated character error rates despite performing accurate OCR.

However, the decrease in WER performance for the OCR models Tesseract and Kraken may be due to multiple files which these models OCR'ed correctly being removed, thus altering their performance distribution. In contrast to this, Idefics showed substantial WER improvement, indicating that files where the model hallucinated (i.e., continuously inserted non-existent words) were removed.

Limitations of VLMs

The poor performance of Idefics, despite being prompted to transcribe the

text in the images, and given the language information, highlights the current limitations of general-purpose vision language models for specialized OCR tasks, such as performing OCR on historical material. Idefics' tendency to hallucinate suggests that while vision language models excel at document understanding, they lack the precision to accurately transcribe the identified text. Instead, it hallucinates and/or does not accurately recognize characters such as "å, ä, ö" present in the specified language. This also suggests that Idefics has not been trained on a variety of languages and/or source material from different time periods. Thus, despite advances in general AI, purpose-built OCR models still remain superior for transcribing source material with varying geographic, typographic, and historical characteristics.

Incremental improvements in OCR for historical material

The marginal improvement in OCR between Tesseract version 4.1.3 and Tesseract version 5.5.0 (a 0.3% difference in CER) suggests a limited cost-benefit of upgrading the OCR system to the latest version. It may also suggest performance plateaus, but that should be investigated further. However, organizations performing OCR may yield higher rewards by developing specialized OCR systems for historical material in their language. It is also possible that Tesseract may benefit substantially by training on different typefaces with different languages.

Architectural differences & Impact on OCR Performance

The results provide valuable insights into how different neural network architectures perform on historical OCR tasks where the typeface, language and spelling may vary. The performance between the LSTM-based models (Tesseract, Kraken, and DocTR) and vision-language models (Idefics) give insight into important considerations for historical document processing.

Tesseract, Kraken and DocTR all utilize bidirectional LSTM architectures to recognize the identified text areas. Yet the models displayed a variation in performance. Kraken, especially Kraken German, performed well for certain time periods in the data with 7.6% CER overall performance. Tesseract performed the best for the majority of the time periods with an overall performance of 6.7%. Although the OCR models share architecture, DocTR performed significantly worse (13.1% CER). The superior performance of Tesseract and Kraken German compared to DocTR indicates that bidirectional LSTM architectures can be highly effective for historical OCR when

properly trained and configured towards the source material.

Most notably, Idefics, presenting cutting-edge vision-language modeling with transformer-based architecture, performed poorly on the source material (with 56.3% CER) despite its advanced multimodal capabilities. Its tendency to hallucinate text and generate repetitive outputs reveal its current limitations in performing OCR. It would be interesting to assess if certain text lengths affect Idefics tendency to hallucinate, which will be proposed in the future work section. From Idefics' performance, it is clear that Idefics has been trained on non-Swedish text and trained on more modern material as the performance in CER for Idefics was 115.4% and 22.8% for the time periods 1818-1859 and 1980-2018 respectively. Idefics performance suggests that general-purpose AI models, despite their impressive capabilities in understanding content and context, cannot yet replace specialized OCR systems.

Furthermore, all OCR models showed varying performance across different time periods. While architectural choice matters, training OCR models on specific source material is equally important.

9.2 Post-OCR model transferability

Output-matched post-OCR models outperform cross-training

One result is that for the post-OCR model transferability, every post-OCR model benefitted the most from correcting the OCR output from the same model. Tesseract achieved a 13% improvement in CER and 29% improvement in WER when corrected by a post-OCR model trained on Tesseract output. DocTR showed a remarkable 36% improvement in CER and staggering 58% improvement in WER with its matching post-OCR model. Similar results are found for Idefics which yielded a 56.5% and 65.5% improvement in CER and WER respectively. Kraken German also saw improvements, with 8.5% improvement in CER and 34% improvement in WER. These results across all OCR models suggest that each OCR model produces distinct, systematic error patterns that can be learned and addressed by post-OCR models specifically trained on correcting those error patterns.

Language improvements with post-OCR

The CER and WER results for post-OCR models trained on their own out-

put, combined with findings that the majority of the models improved error patterns related to Swedish special characters (å, ä, ö), suggest that even though the OCR models are not specifically trained on Swedish language, their output can still be corrected with great accuracy to proper Swedish. This indicates the potential performance these OCR models could achieve if trained on Swedish historical material.

Suboptimal output into superior performance

A remarkable finding is that, even though DocTR performed second to last in the general OCR assessment, the post-OCR correction model corrected its erroneous output to such a degree that the final result achieved better performance than the best baseline OCR model and its post-OCR corrected output. This suggests that post-OCR correction can transform moderate-quality OCR output into high-quality results, provided that the error patterns are repeatable and learnable.

Formatting impacts transferability

Idefics yielded a 56.5% improvement in CER and a 65.5% improvement in WER, which is the highest percentage improvement for all post-OCR models. This was achieved by the post-OCR model trained on Idefics, where both the training data and testing data were entirely stripped of new lines, which Idefics otherwise adds to its OCR output. These improvement changes are not present in the post-OCR models trained on DocTR, Tesseract or Kraken German when applied to Idefics (unfiltered) OCR output. Instead, those post-OCR models achieved a CER and WER improvement of 1.2-3.4% and 2.2-6.3% improvement respectively. This limited improvement suggests that formatting inconsistencies between training and testing data significantly affect post-OCR models' ability to correct source material.

Cross-model effectiveness

The transferability between different OCR models show notable asymmetries. DocTR output when trained on other post-OCR models showed a surprisingly good results when the post-OCR model trained on Idefics corrected the output, achieving a 27% CER improvement and 49% WER improvement on DocTR's output. This suggests that the output from DocTR shares common error patterns with other OCR systems, making its output ideal for cross-model correction.

In contrast, Kraken German and Tesseract showed limited cross-model transferability, with post-OCR models providing minimal CER improvements while still yielding moderate WER improvements. This may indicate that these models produce distinctive error patterns that are less compatible with post-OCR models trained on other source material, or it could suggest that the output quality from Kraken and Tesseract contains less systematic error patterns suitable for cross-model correction.

The differences in CER & WER improvement

The results from post-OCR training show varying improvement in CER but consistently positive improvement in WER. This suggests that even when character-level error correction may be minimal, word-level correction consistently benefits from post-OCR processing. The contrasting performance between CER and WER likely stems from the fact that while some characters are being corrected to create more complete words, those words that remain incorrect may become even more incorrect after post-OCR processing, resulting in improved word recognition overall despite potentially increased character-level errors in remaining problematic words.

9.3 Post-OCR with synthetic data

The results from training with different synthetic data demonstrate that the effectiveness of a post-OCR correction model is highly dependent on the domain of the synthetic training material.

Domain specific synthetic data results

The newspaper synthetic post-OCR model (S^{news}) achieved a slight improvement in WER (-1.9%) but a modest performance decrease in CER (+9.2%). The performance decrease in CER may be due to the blur, text distortions and stroke width variations within the synthetic data, which may make it harder to properly recognize certain characters. If one assesses the underlying changes, the post-OCR model corrects Fraktur errors to a great extent, but also introduces new errors that were not previously present in the data. This suggests that when synthetic data closely matches the source material, post-OCR models can learn certain meaningful correction patterns, but with limitations.

Non-domain specific synthetic data results

The Wikipedia synthetic post-OCR model (S^{wiki}) demonstrated a severe performance degradation, with CER and WER worsened by 197% and 127% respectively. The hypothesis for the synthetic Wikipedia data was that more data would result in a better understanding of the Swedish language, and as such, can more accurately correct the erroneous output. However, the results from the trained post-OCR model on Wikipedia data shows that the post-OCR model learned to insert characters where there should not have been any insertions, and also increased the number of substitutions. It introduces many new errors previously not present in the OCR output. One such example is the introduction of substituting f to k . This implies that the large number of training data resulted in many learned patterns, which are not present in the source material. These over-correction patterns suggest that synthetic data trained on material from another domain (not Swedish historical newspaper pages) can teach the post-OCR models to apply “corrections” which are not compatible with the target material, leading to a performance degradation.

Mixed domain specific synthetic data results

The combined approach yielded the most promising results, namely the $S^{original+wiki}$ post-OCR model. It successfully decreased word level errors by 15.9% albeit with a slight performance degradation on the character level (+6.1%). This suggests that combining synthetic data sources can help balance domain specific biases (i.e. the ones present in S^{wiki}) and create more robust correction models. The error analysis shows that the combined model successfully learned to correct historical typography errors (94.9% reduction in $f \rightarrow f$ errors, 97% reduction in $k \rightarrow f$ errors) while avoiding the severe over-correction patterns observed in the Wikipedia-only model.

Comparison and recommended approach

The contrasting performance between the three models, S^{news} , S^{wiki} and $S^{original+wiki}$ suggests that while the newspaper synthetic data was more domain-appropriate for correcting the OCR output, the synthetic Wikipedia data likely provided more linguistic diversity, albeit at the cost of learning patterns not present in the historical material. The performance of the combined approach in $S^{original+wiki}$ implies that it both combines domain matching but also achieves sufficient linguistic understanding to prevent overfitting specific error patterns. The combined post-OCR model’s ability to reduce

both insertion errors (-19.8%) and substitution errors (-30.8%) indicates that a balanced synthetic training data set can produce meaningful corrections.

These findings demonstrate that while synthetic data training for post-OCR correction is feasible, success requires careful domain matching and multi-source approaches to achieve meaningful improvements without introducing systematic errors.

10 Conclusion

This study evaluated the performance of recent OCR engines when applied to Swedish historical newspaper material spanning 200 years (1818-2018) and investigated the effectiveness of post-OCR correction models, including their transferability when applied to different OCR engines' outputs and examined the feasibility of training post-OCR models exclusively on synthetic data.

Comparative performance of recent OCR models

The comparative analysis revealed that Tesseract version 5.5.0 achieved the best overall performance with 6.7% CER and 22.4% WER. Tesseract was closely followed by Kraken German that yielded an overall performance of 7.6% CER and 26.5% WER even though Kraken German did not recognize the Swedish character "å". This, along with the fact that Kraken German outperformed Tesseract for a certain time period in the source material suggests that organizations and institutions may benefit from developing specialized OCR models to target certain typography styles and languages, rather than applying a single solution across all time periods and typefaces.

Furthermore, DocTR achieved 13.1% CER and the vision-language model Idefix performed poorly at 56.3% CER. Even though general-purpose AI solutions like the vision-language model Idefix show promise in OCR, its tendency to hallucinate and generate repetitive words signify the limitations of cutting-edge vision-language models application to OCR, although it should be evaluated in greater detail in which situations Idefix has a tendency to hallucinate.

Even though Tesseract, Kraken, and DocTR have similar architecture components (bidirectional LSTM) for recognizing text, their results differ. This

suggests that language support, model optimization, and training data similar to the target domain are critical factors to ensure optimal OCR performance. This is also illustrated with the results from Idefics, that likely has been trained on recent source material with its prominent performance difference between the time periods 1818-1859 and 1980-2018 with a CER performance of 115.4% and 22.8% respectively.

Furthermore, the performance difference across time periods revealed persistent challenges in processing historical documents, with even the best-performing OCR model Tesseract showing an eight-fold increase in character error rates when comparing the oldest material (1818-1859) to the most recent material (1980-2018). This varying performance underscores that modern OCR engines still remain optimized for processing modern material, which highlights the need for specialized approaches, tailored to historical collections, their spelling variations and character appearance.

Cross-model post-OCR correction transferability

The post-OCR correction models demonstrated substantial improvement when the post-OCR model was matched to their corresponding OCR engine outputs. DocTR achieved remarkable post-OCR results, surpassing the best baseline OCR model Tesseract and its post-OCR performance and positioning DocTR as the OCR engine, when applied with post-OCR, can accurately correct the limitations of its OCR output and yield the best overall CER results of 9%. This demonstrates that correction models can effectively compensate for systematic OCR errors when the error patterns are learnable and consistent. This also suggests that language limitations in the OCR output can easily be corrected.

The cross-model transferability revealed some asymmetries, with the OCR output from DocTR being the most compatible for post-OCR correction, where all models achieved WER improvements between 21.7% to 48.9%, indicating that DocTR's output contains learnable patterns for all post-OCR correction models. In contrast, Tesseract showed limited transferability between the different post-OCR models. This limited improvement may be due to Tesseract's already high baseline accuracy, leaving less room for correction.

Effectiveness of synthetic post-OCR correction models

The synthetic data experiments revealed that a combined approach, combin-

ing the original source material with synthetic non-domain data, achieved the best performance for post-OCR correction, with a WER performance increase of 15.9%. The domain-accurate synthetic data (newspaper) achieved a slight WER improvement, while non-domain synthetic data (Wikipedia) caused severe performance degradation, resulting in an over-correction of the OCR output. The combined original and synthetic approach yielded the most promising results, which demonstrates that multi-source training data can balance domain patterns with a broader linguistic diversity to create viable correction models. However, it should be noted that the performance still falls short from performing post-OCR correction without the usage of synthetic data.

11 Future work

OCR Models trained for historical Swedish material

To accurately assess the OCR models performance, it would be interesting to fine-tune the OCR models on Swedish material. This would give deeper insight into their comparative performance and would better reflect their architectural differences.

Furthermore it would be interesting to fine-tune an OCR model on Swedish historical material, displaying various typefaces, such as Fraktur, Antiqua, and Schwabacher in the training data.

On that note, it would be interesting to provide Tesseract and other models that support it with additional dictionaries, containing Swedish dictionaries from 1800-2000s. This was made by Löfgren and Dannélls, and resulted in Tesseract version 4s performance of 4.9% CER for the same data set employed in this study [6]. I tried to re-create these results with dictionaries, but as only two of the used dictionaries were public, no notable differences were achieved.

Vision-language model investigation

Given Idefics' poor performance despite its advanced capabilities, a future research direction is to investigate which specific conditions trigger the hallucination of Idefics. The hallucination and generation of repetitive words may occur under certain text characteristics - such as at a certain text length.

Investigating Transformers

It would also be interesting to include a transformer model in the OCR comparison assessment, as Transformers have shown promising results for NLP tasks. It would be interesting to assess if Transformers are superior in processing sequential text-image data or if BiLSTMs have an advantage in their bidirectional architectural structure.

Synthetic data ratio for post-OCR

Some research has explored the synthetic data ratio for training post-OCR models. The ratios explored have been at most 2:1 ratio, whereas from this study, for the post-OCR model S^{wiki} , the ratio was 10:1. Further analysis, determining at which ratio the overcorrection occurs and which ratio yields the optimal synthetic data ratio is of interest.

Refined ground truth

As the ground truth used in this thesis in certain files have incomplete transcriptions, it would be interesting if those files could be filtered out entirely from the source material, before processing it with the OCR Engines. Here, perhaps a naive approach could be employed, where files that Tesseract consistently yields high CER/WER scores are removed.

Furthermore, it would also be interesting to employ a classification model to identify and segment these image files, in order to then remove them entirely or handle them differently, perhaps with a scene-detection model or similar (depending on the image contents).

It would also be of interest to use an already refined dataset, to then perform the OCR performance assessment.

Employing context-aware post-OCR models

Certain promising research has been made where the context of the source material is given to the language model, which resulted in a 60% reduction in CER [15]. To provide the historical context to the language model to perform post-OCR is an approach that could heavily impact digitizing historical documents.

Appendix

Annotations in the data

Table 30: Annotation Types and HTML Codes

Annotation type	HTML codes	Description
Typeface annotations	<fr></fr>	For text in Fraktur typeface
	<aq></aq>	For text in Antiqua
	<sw></sw>	For text in Schwabacher
Text formatting annotations	<fr></fr>	For text in Fraktur typeface
	<aq></aq>	For text in Antiqua
	<sw></sw>	For text in Schwabacher
	<i></i>	Italic text
	<sc></sc>	Small capitals
	<big></big>	Text larger than body text
		Text smaller than body text
	<init></init>	Decorative initials spanning several lines
Ligature annotations	_ (underscore)	Used between characters to mark ligatures
	t_z	tz ligature
	c_t	ct ligature
	s_t	st ligature
	o_e	oe ligature
	a_e	ae ligature
	m_m / M_M	m/M with doubling sign
Special characters	ß	sz ligature
	þ	thorn
	¶	pilcrow
	§	section
	7	Tironian et sign
	@	at sign
	#	hashtag
	%	percent sign
	‰	per mille sign
	©	copyright sign
	*	asterisk
	f	long s
Other transcription rules	Line breaks and hyphens	Preserved as in original
	Blank lines	Represented by two consecutive line breaks
	Fractions	Written with slash (e.g., 3/8)
	Double hyphen in Fraktur	Rendered as = (equals sign)

Additional explanations of OCR models

Kraken

Kraken is a turn-key open-source OCR system that is optimized for historical and non-Latin script material. It has a three-stage pipeline consisting of preprocessing (i.e. binarization), segmentation (where layout analysis and line detection is executed) and the text recognition stage. The OCR engine is specifically designed for handling historical documents and various writing systems. Kraken was first published 2019 and was created by Benjamin Kiessling [11].

The text recognition of Kraken is essentially a rewritten fork of the OCRopus system, where Kraken has been extended with features that enable the processing of most scripts and writing systems, such as right-to-left and vertical writing support, to name a few [11]. What is interesting about Kraken is not only that it is specifically optimized for historical material, but that it also employs a segmentationless sequence-to-sequence approach, which enables Kraken to recognize whole lines of text, without having to split up words into character-level segmentation to then perform recognition on each letter. Instead, the neural network recognizes whole lines of text [27]. This is a clear distinction from other OCR systems, like *Tesseract* and *Abbyy FineReader* [11]. The architecture of Kraken is visible in Figure 48.

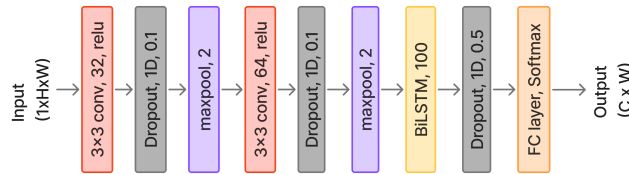


Figure 48: Kraken recognition network architecture

In terms of pre-processing, Kraken performs *binarization* where the images are converted into black and white. For the segmentation stage, Kraken identifies the lines of text by using a “U-Net” to scan the binarized images and find where the lines of texts are located. Then it cleans up this text-highlight-map by smoothing out rough edges (with Gaussian smoothing), making sure

it only keeps the strongest text signals (called hysteresis thresholding) upon which it then creates the lines that follow the text (called skeletonization). This enables Kraken to identify where text is in the image, and also to maintain which order to read the text in.

Once Kraken knows where the text is, Kraken recognizes the text by utilizing a hybrid convolutional and recurrent neural network (CRNN). The convolutional layers are spotting the visual patterns (like letters) and the recurrent layers are handling the sequences, such that words can be properly formed from the visual patterns. This architecture enables Kraken to process an image of a text line and directly output the characters, without having to recognize each letter one by one. Kraken also uses dropout layers, where certain information is let out, in order to ensure Kraken is not overly reliant on certain features and patterns. Similarly, it also employs early stopping to ensure Kraken stops the training before it starts memorizing instead of learning [11].

Kraken has 27 pre-trained OCR models, each model specialized on a certain time period, language and document type. The different pre-trained OCR models are:

- Kuzushiji (Japanese historical script)
- Segmentation model for historical Samaritan Manuscripts (single column)
- McCATMuS - Transcription model for handwritten, printed and type-written documents (16th-21st century)
- Latin graphematic transcription model (11th century manuscripts)
- Latin expanded transcription model (11th century manuscripts)
- Latin Incunabula and Early Prints
- HTR-United model (17th-21st century)
- CATMuS-Print (Tiny) - Diachronic model for French prints and other languages

- CATMuS-Print (Large) - Diachronic model for French prints and other languages
- German prints model (trained from several datasets)
- Experimental printed Syriac model
- French 20th century handwriting model (Lucien Peraire)
- German manuscripts model (trained from several datasets)
- Fraktur model (trained from Austrian Newspapers dataset)
- Medieval manuscript model (8th-16th century, GalliCorpora, CREMMA Medieval)
- Gallicorpora+ ancient prints (Literature)
- Printed Persian Base Model (OpenITI Corpus)
- Printed Ottoman Base Model (OpenITI Corpus)
- Printed Arabic Base Model (OpenITI Corpus)
- Printed Arabic-Script Base Model (OpenITI Corpus)
- Cremma-Medieval Old French Model (Literature)
- LECTAUREP Contemporary French Model (Administration)
- Medieval Hebrew manuscripts in Sephardi bookhand
- Medieval Hebrew manuscripts in Italian bookhand
- Medieval Hebrew manuscripts in Ashkenazi bookhand
- Medieval Hebrew manuscripts (general)
- Generalized model for English printed text

DocTR

The company Mindee developed and maintains DocTR, a turn-key OCR system powered by Tensorflow 2 and PyTorch. DocTR performs both text detection and text recognition. For each step, DocTR has various model selections.

Text detection models

The supported text detection models are DBNet, LinkNet and FAST [32]. These detection models will be described and motivated to why the certain model selection was made.

LinkNets uses an encoder for the text detection, which learns the important visual features and also provides the decoder with detailed spatial information, i.e. where the edges of text areas are precisely located. This helps the decoder to create a much more accurate segmentation map and it does this with fewer computations, in comparison to VGG or earlier ResNets used for similar tasks [33]. The text detection model FAST on the other hand, has an architecture based on a minimal kernel representation, which means that instead of trying to make the network predict the entire text region direction, it instead predicts a much smaller representation of it (a kernel). To then find the whole shape of the text area it uses GPUs as a post-processing step, that essentially takes these minimalist kernels and dilute them outwards, expanding them until they cover the text area [34]. Lastly, the text detection model DBNet uses a flexible binarization to find the text areas. targets the binarization step to find the text areas. Traditionally, binarization was a separate step where the network yields a probability map of where text is likely to be located (pixel by pixel) and then pick an arbitrary rule, i.e. any pixel with more than 0.7 probability is likely text. However, a single fixed threshold might not work equally as well for all the pixels in the image. DBNet targets this problem by using an adaptive threshold map, with different thresholds for each pixel. This preprocessing step is more flexible as it allows the thresholding to adapt to local conditions in the image [23].

Text recognition models

The supported text recognition models are CRNN, SAR, ViTSTR, MASTER, and PARSeq [32]. CRNN is an end-to-end sequence to sequence-to-sequence model, which is a hybrid convolutional recurrent neural network.

First, the image is fed to the convolutional layer that extracts the key visual information and outputs the feature map. The recurrent layers take the feature map and process it sequentially. The recurrent layers are usually LSTMs and before the LSTMs there usually is a max pooling step. The transcription layer then takes the sequence of outputs that the RNN generated and maps it onto the actual text sequence, which it does by Connectionist temporal classification (CTC) [24]. CRNN is designed as inherently good for sequential data and shows a decent level of robustness towards minor distortions etc. However, RNNs may have problems with long range dependencies (i.e. relations between words that are far apart), in which transformers generally perform better.

One transformer based model is SAR, that has an encoder-decoder architecture and uses a 2D attention mechanisms. SAR extracts a 2D feature map from the input image, then an LSTM encoder processes it with the purpose of yielding a holistic feature representation, that summarizes the entire image context. Then the LSTM decoder generates the output text, character by character but at each step, where it is deciding on the next character, a 2D attention module lets it look back and focus on the most relevant parts of that entire 2D feature map [35].

ViTSTR is a vision transformer for scene text detection. Its architecture is a standard vision transformer architecture which it adapts to text recognition by chopping the images into patches and treat these patches like words in a sentence, which is then fed into a standard transformer encoder. The output from the encoder is then used to predict the character sequence [36].

Multi-aspect self-attention network (MASTER) is another transformer-based recognition model. Instead of using LSTMs it utilizes self-attention within its architecture. The encoder in master uses multi aspect global attention in which the encoder learns to make connections not just between the image features and the output text, but also within the input features themselves (it understands the inherent relationships between the image features). The multiaspect part of its name means that it is looking at different kinds of spatial relationships. It also has a global context, as it considers the entire image and understands how different parts of the image relate to each other simultaneously. This could potentially be useful for complex layouts or fonts where relationships are not always linear. As it uses a transformer architec-

ture, it can process part of the input features in parallel. Which makes it computationally efficient compared to RNNs. It has good performance on regular text [37].

Another interesting model apart from MASTER is PARSeq, which stands for Permuted autoregressive sequence model. What is interesting about PARSeq is that instead of using autoregressive modelling, where each character is predicted based on the previous characters, it instead has been trained using permutations, or shuffled orders. This forces the model to learn more robust relations between characters, as it cannot just rely on the immediately preceding character, it instead has to understand the character's identity based on a more varied set of neighbors, depending on the permutation. Furthermore, what is interesting about PARSeq is its feature *iterative refinement*, in which the model makes an initial prediction of the character and can then return and refine certain characters. As PARSeq is the chosen text recognition model, its architecture will be explained in further detail. It consists of a 12-layer Vision Transformer to process the input image, in which the image is divided into patches and into tokens, and it processes the entire image through self-attention mechanisms and outputs feature tokens to the decoder. The Decoder is a one layer Visio-lingual Decoder that has three main components. It has a Context-Position Attention module which takes position tokens as queries, context embeddings as keys and values and uses attention mask based on permutations during training. This allows parallel prediction rather than sequential decoding. For the Image-Position Attention Module, it takes the output from the context-position attention as queries and uses the encoded image features as keys and values and connects the visual information to the textual context. Lastly, PARSeq has a Multi-Layer Perceptron which processes the outputs from the attention mechanisms and helps transform the representations before the final linear layer performs the character predictions [38].

Previous OCR results - external results with word lists

Table 31: OCR Performance Metrics Across Time Periods (Baseline Models)

Period	Model	CER	WER	count of paragraphs
Overall	ABBY1	0.041	0.169	41068
	Abby-Tesseract1	0.040	0.158	41068
	Tesseract1	0.049	0.201	27178
1818-1859	ABBY1	0.111	0.415	2527
	Abby-Tesseract1	0.111	0.399	2527
	Tesseract1	0.108	0.408	2131
1860-1899	ABBY1	0.054	0.227	12601
	Abby-Tesseract1	0.052	0.215	12601
	Tesseract1	0.057	0.239	8825
1900-1939	ABBY1	0.031	0.130	11498
	Abby-Tesseract1	0.028	0.115	11498
	Tesseract1	0.040	0.181	5961
1940-1979	ABBY1	0.020	0.076	9390
	Abby-Tesseract1	0.019	0.071	9390
	Tesseract1	0.031	0.122	6573
1980-2018	ABBY1	0.012	0.052	5052
	Abby-Tesseract1	0.012	0.050	5052
	Tesseract1	0.019	0.073	3688

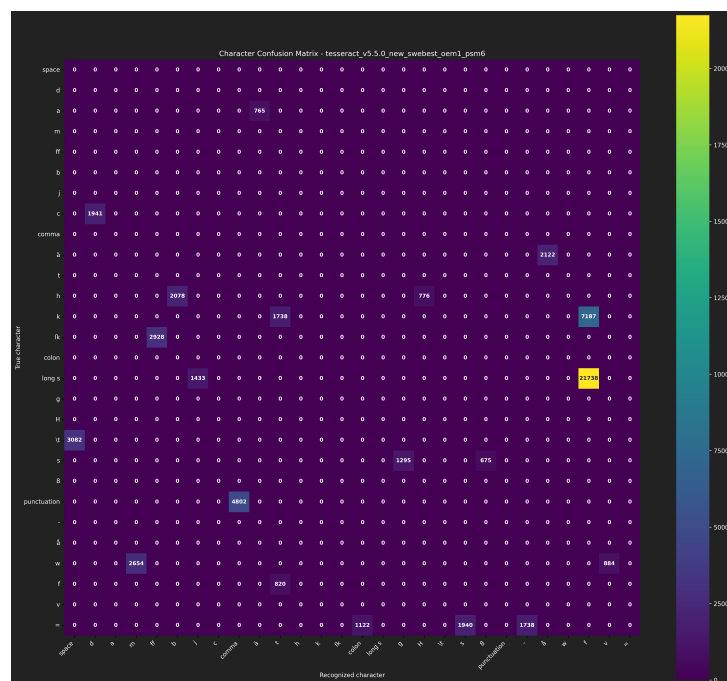


Figure 49: Confusion matrix - top 20 errors. Tesseract v5.5.0

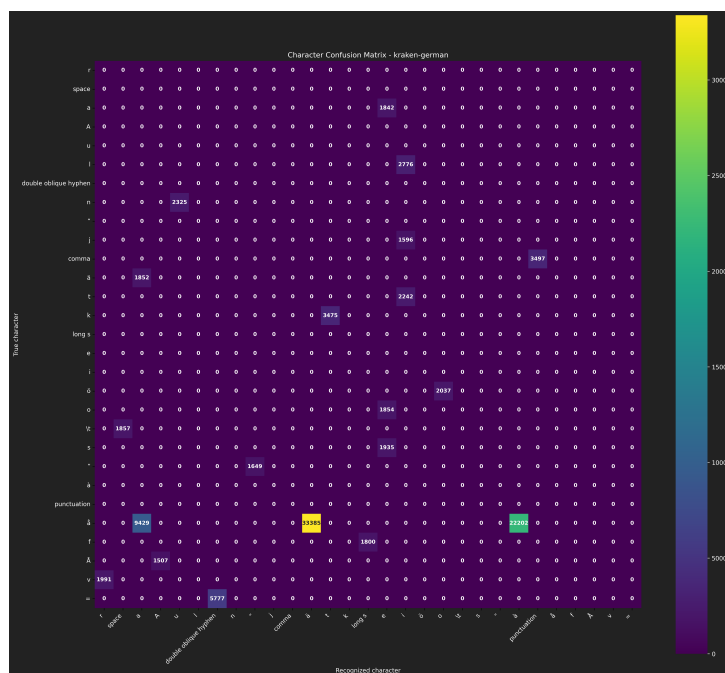


Figure 50: Confusion matrix - top 20 errors. Kraken-German

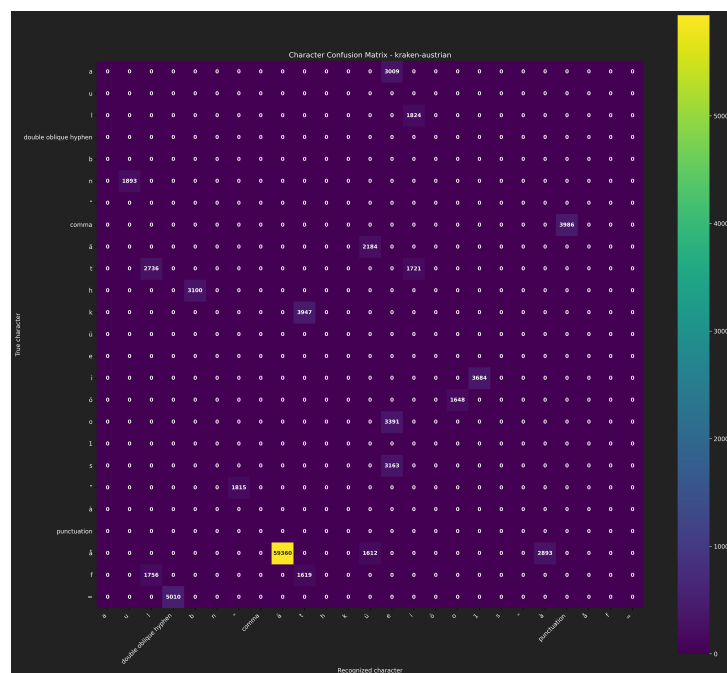


Figure 51: Confusion matrix - top 20 errors. Kraken-Austrian

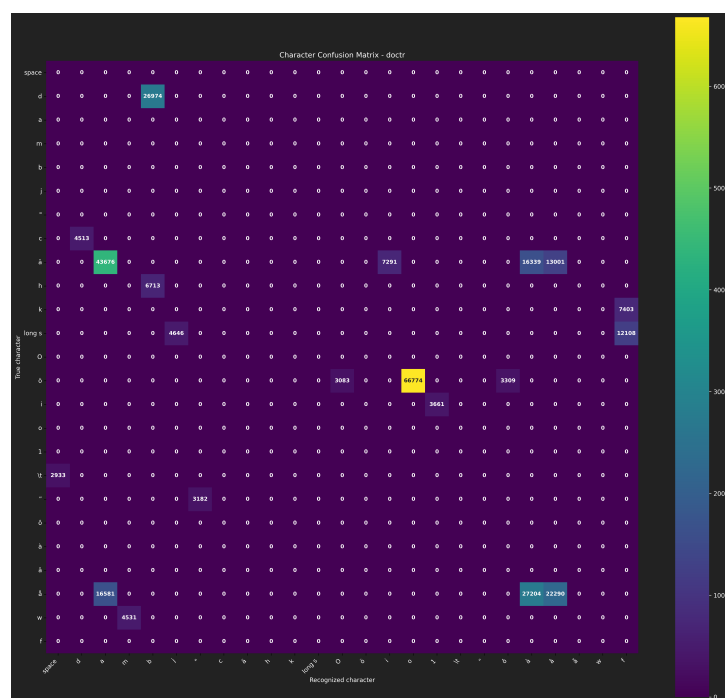


Figure 52: Confusion matrix - top 20 errors. DocTR

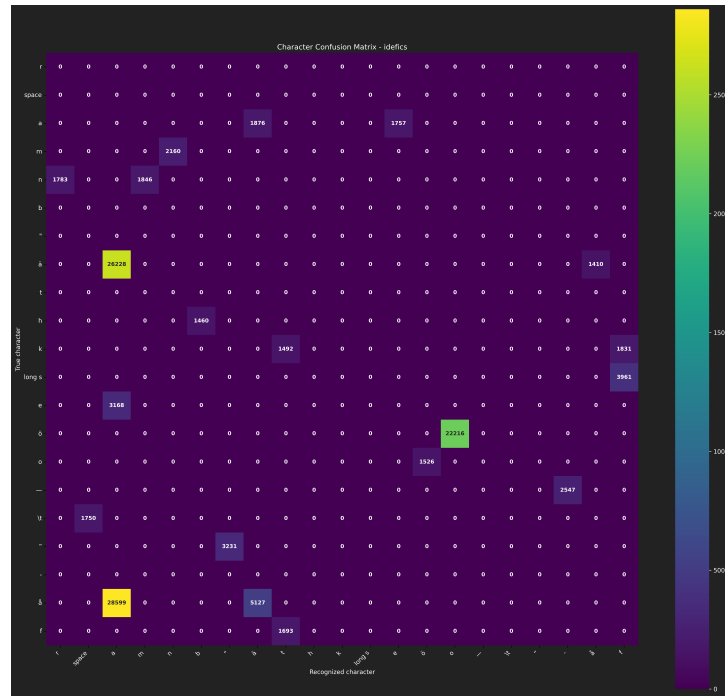


Figure 53: Confusion matrix - top 20 errors. Idefics

Usage of generative AI

All code files except the ones specifically mentioned have been created in collaboration with Claude 3.7 Sonnet. The process has usually been that I start the code script, ask for specific functions and incorporate them after assessment and potentially debug.

Furthermore, for the report, Claude has been used to create accurate references to the reference list (i.e. bib sources in Overleaf) and updating table values. I have cross-checked both the reference list and the table values to ensure they display correct information. Claude has also been used in the last step in order to highlight incorrectly spelled words that I missed during my read-throughs of the report.

References

- [1] M. Li, T. Lv, J. Chen, L. Cui, Y. Lu, D. Florencio, C. Zhang, Z. Li, and F. Wei, “Trocr: Transformer-based optical character recognition with pre-trained models,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, no. 2, 2023, pp. 13 094–13 102.
- [2] A. Lundberg and M. Torstensson, “Deep learning for post-ocr error correction on Swedish texts,” Master’s thesis, Chalmers University of Technology and University of Gothenburg, Gothenburg, Sweden, 2021.
- [3] M. Koistinen, K. Kettunen, J. Kervinen, P. Paroubek, M. Kubis, and Z. Vetulani, “How to improve optical character recognition of historical finnish newspapers using open source tesseract ocr engine – final notes on development and evaluation,” in *Human Language Technology. Challenges for Computer Science and Linguistics*, ser. Lecture Notes in Computer Science. Switzerland: Springer International Publishing AG, 2020, vol. 12598, pp. 17–30.
- [4] D. Dannélls, L. Björk, T. Johansson, and O. Dirdal, “A two-OCR engine method for digitized Swedish newspapers,” in *Selected papers from the CLARIN Annual Conference 2020*, ser. Linköping Electronic Conference Proceedings, vol. 180, 2021, pp. 65–74.
- [5] National Library of Sweden, “Det här gör vi,” Kungliga biblioteket, 2024, accessed on 2024-10-29. [Online]. Available: <https://www.kb.se/om-oss/det-har-gor-vi.html>
- [6] V. Löfgren and D. Dannélls, “Post-ocr correction of digitized Swedish newspapers with ByT5,” in *Proceedings of LaTeCH-CLfL 2024*. Association for Computational Linguistics, March 2024, pp. 237–242, published March 22, 2024.
- [7] S. Persson, “Ocr post-processing of historical Swedish text using machine learning techniques,” Master’s thesis, 2019. [Online]. Available: <https://hdl.handle.net/20.500.12380/300605>
- [8] G. Pettersson, *Svenska språket under sjuhundra år : en historia om svenskan och dess utforskande*, 2nd ed. Lund: Studentlitteratur, 2005.

- [9] Svenska Akademiens Ordbok, “Schwabacher,” <https://www.saob.se/artikel/?seek=schwabacher&pz=6>, accessed: 2025-04-15.
- [10] O. Trier and T. Taxt, “Evaluation of binarization methods for document images,” *IEEE transactions on pattern analysis and machine intelligence*, vol. 17, no. 3, pp. 312–315, 1995.
- [11] B. Kiessling, “Kraken: Open source ocr engine,” 2024, apache-2.0 licensed. [Online]. Available: <https://github.com/mittagessen/kraken>
- [12] R. C. Carrasco, “An open-source ocr evaluation tool,” in *Proceedings of the First International Conference on Digital Access to Textual Cultural Heritage*. New York, NY, USA: ACM, 2014, pp. 179–184.
- [13] M. Brandt Skelbye and D. Dannélls, “OCR processing of Swedish historical newspapers using deep hybrid CNN–LSTM networks,” in *Proceedings of the International Conference on Recent Advances in Natural Language Processing (RANLP 2021)*. Held Online: INCOMA Ltd., Sep. 2021, pp. 190–198. [Online]. Available: <https://aclanthology.org/2021.ranlp-1.23>
- [14] S. Drobac and K. Lindén, “Optical character recognition with neural networks and post-correction with finite state methods,” *International Journal on Document Analysis and Recognition (IJDAR)*, vol. 23, pp. 279–295, 2020.
- [15] J. Bourne, “Scrambled text: fine-tuning language models for ocr error correction using synthetic data,” *International journal on document analysis and recognition*, 2025.
- [16] I. Gruber, L. Pícek, M. Hlaváč, P. Neduchal, M. Hruží, H. Moosaei, P. M. Pardalos, and M. Hladík, “Improving handwritten cyrillic ocr by font-based synthetic text generator,” in *Dynamics of Information Systems*, ser. Lecture Notes in Computer Science. Cham: Springer Nature Switzerland, 2023, pp. 102–115.
- [17] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016, <http://www.deeplearningbook.org>.

- [18] Y. Bengio, P. Simard, and P. Frasconi, “Learning long-term dependencies with gradient descent is difficult,” *IEEE transactions on neural networks*, vol. 5, no. 2, pp. 157–166, 1994.
- [19] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems*, I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, Eds., vol. 30. Curran Associates, Inc., 2017, pp. 5998–6008.
- [20] R. Smith, “An overview of the tesseract ocr engine,” vol. 2, 10 2007, pp. 629 – 633.
- [21] S. Weil, R. Smith, Z. Podobny, and Tesseract OCR Team, “Tesseract: Open source ocr engine,” Tesseract OCR, major version 5 is the current stable version. [Online]. Available: <https://github.com/tesseract-ocr/tesseract>
- [22] Tesseract OCR Team, *Tesseract OCR: User Manual*, Tesseract OCR, 2025, version 5.x. Open source text recognition (OCR) Engine under the Apache 2.0 license. [Online]. Available: <https://tesseract-ocr.github.io/tessdoc/>
- [23] M. Liao, Z. Wan, C. Yao, K. Chen, and X. Bai, “Real-time scene text detection with differentiable binarization,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, no. 7, 2020, pp. 11 474–11 481.
- [24] B. Shi, X. Bai, and C. Yao, “An end-to-end trainable neural network for image-based sequence recognition and its application to scene text recognition,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 39, no. 11, pp. 2298–2304, 2017.
- [25] H. Laurençon, A. Marafioti, V. Sanh, and L. Tronchon, “Building and better understanding vision-language models: insights and future directions.” 2024.
- [26] M. Noyan and E. Beeching, “Vision language models explained,” April 2024, online blog post. [Online]. Available: <https://huggingface.co/blog/vision-language-models>

- [27] B. Kiessling, G. Kurin, M. Miller, and K. Smail, “Advances and limitations in open source arabic-script ocr: A case study,” *Digital Studies/Le champ numérique*, vol. 11, no. 1, p. 8, 2021. [Online]. Available: <https://arxiv.org/pdf/2402.10943>
- [28] B. Kiessling, “Kraken - an universal text recognizer for the humanities.” Université PSL, France and Leipzig University, 2019, paper presented at a digital humanities conference or workshop.
- [29] Tesseract OCR, “Trained models with fast variant of the “best” LSTM models + legacy models,” 2024, these language data files work with Tesseract 4.0.0 and newer versions, having models for both legacy tesseract engine (–oem 0) and the LSTM neural net based engine (–oem 1). The LSTM models have been updated to integerized versions of tessdata_best, making them faster but potentially less accurate. [Online]. Available: <https://github.com/tesseract-ocr/tessdata>
- [30] Tesseract OCR Team, *Command Line Usage - Tesseract OCR*, Tesseract OCR, 2025, documentation for Tesseract 5.x. Lead developer: Stefan Weil. Maintainer: Zdenko Podobny. Accessed on May 4, 2025. [Online]. Available: <https://tesseract-ocr.github.io/tessdoc/Command-Line-Usage.html>
- [31] KB Lab, “Post-ocr correction,” <https://github.com/kb-labb/post-ocr-correction>, 2024. [Online]. Available: <https://github.com/kb-labb/post-ocr-correction>
- [32] Mindee, “doctr: Document text recognition,” <https://github.com/mindee/doctr>, 2021.
- [33] A. Chaurasia and E. Culurciello, “Linknet: Exploiting encoder representations for efficient semantic segmentation,” in *2017 IEEE Visual Communications and Image Processing (VCIP)*. IEEE, 2017, pp. 1–4.
- [34] Z. Chen, J. Wang, W. Wang, G. Chen, E. Xie, P. Luo, and T. Lu, “Fast: Faster arbitrarily-shaped text detector with minimalist kernel representation,” *arXiv preprint arXiv:2111.02394*, 2021.
- [35] H. Li, P. Wang, C. Shen, and G. Zhang, “Show, attend and read: A simple and strong baseline for irregular text recognition,” in *Proceedings*

- of the AAAI Conference on Artificial Intelligence*, vol. 33, no. 1, 2019, pp. 8610–8617.
- [36] R. Atienza, “Vision transformer for fast and efficient scene text recognition,” in *International Conference on Document Analysis and Recognition*. Springer, 2021, pp. 319–334.
- [37] N. Lu, W. Yu, X. Qi, Y. Chen, P. Gong, R. Xiao, and X. Bai, “Master: Multi-aspect non-local network for scene text recognition,” *Pattern Recognition*, vol. 117, p. 107980, Sep. 2021. [Online]. Available: <http://dx.doi.org/10.1016/j.patcog.2021.107980>
- [38] D. Bautista and R. Atienza, “Scene text recognition with permuted autoregressive sequence models,” in *European Conference on Computer Vision*. Springer, 2022, pp. 178–196.